Machine Learning Lecture Notes
2025/2026

João Pires (116478)
Instituto Superior Técnico, University of Lisbon, Portugal

26 of October, 2025

1. Machine Learning

Existem muitos problemas em que a relação entre certas variáveis pode ser descrita por uma equação matemática, por exemplo. Como é o caso das equações de Maxwell ou das leis de Newton. Mas existem certos problemas em que não há uma relação concreta entre as variáveis, como é o caso de prever a sobrevivência de um paciente num hospital, reconhecer uma face ou até traduzir uma frase. Nestes casos não existem equações matemáticas que os descrevam, portanto, a melhor coisa a fazer é aprender a partir dos dados disponíveis, daí vem o nome Machine Learning (ML).

Por exemplo, no que toca ao caso do hospital. Temos vários pacientes que deram entrada no hospital com certas características, como sintomas, idade, histórico de doenças, etc., e sabemos se saíram vivos ou mortos. Não temos uma relação matemática que descreva se o paciente vai sair morto ou vivo, mas podemos aprender pelos exemplos passados e, assim, quando um paciente der entrada no hospital, e tendo em conta todas estas variáveis, podemos prever se sai vivo ou morto. Isto é um exemplo de supervised learning, aprender uma relação entre as entradas (features) e as saídas (labels).

Hoje em dia vivemos numa sociedade em que existem muitos dados a serem gerados a toda a hora, seja a partir das redes sociais, seja a partir de sensores, transações online, etc. E dado este flow incalculável de dados, o Machine Learning tornou-se indispensável nos dias de hoje. Quanto mais dados tivermos, mais competente torna-se o nosso modelo de ML, dado que tem noção de mais possibilidades e tipos de ocorrências de interesse. ML é utilizado em uma enorme variedade de contextos, desde a predição e análise temporal até à deteção de anomalias. Um marco importante foi o surgimento das redes neuronais profundas, Deep Learning, que servem para tarefas 'mais complicadas'.

Temos 3 tipos principais de categorias de problemas de aprendizagem: a aprendizagem supervisionada, a aprendizagem não supervisionada e a aprendizagem por reforço. Na aprendizagem supervisionada, o algoritmo recebe pares de dados (x, y) onde x representa a entrada (input) no modelo e y a saída (output) desse respetivo x, e aprende uma função $f(x) \approx y$. Na aprendizagem não supervisionada, apenas os inputs são fornecidos e o objetivo é descobrir estrutura nos dados (agrupamentos, distribuições, etc.). E por fim, temos a aprendizagem por reforço, onde um agente aprende por tentativa e erro, recebendo recompensas ou penalizações do ambiente.

Para exemplos introdutórios, dois ajudam a fixar as diferenças. Na regressão, podemos prever o preço de um apartamento com base na sua área. Na classificação, podemos distinguir um atum de um peixe-espada a partir do comprimento e do volume. Em ambos estes casos, na regressão e classificação, temos as entradas/inputs (features), as saídas/outputs (outcome) e o modelo preditivo (predictor).

O objetivo é encontrar uma função $f : \mathbb{R}^p \to \mathbb{R}$ (ou para um conjunto discreto de classes) tal que $\hat{y} = f(x)$ se aproxime do valor verdadeiro y, onde $\hat{y}$ é a predição.

Dado um conjunto de treino $T = \{(x^{(i)}, y^{(i)})\}_{i=1}^{n}$, procuramos a função f que minimiza a discrepância entre y e $\hat{y}$. Essa discrepância é medida por uma função de perda/custo $L(y, \hat{y})$ (loss function), e o objetivo é minimizar a soma (ou média) dessas perdas no treino. Formalmente:

$$\hat{f} = \arg\min_{f} \sum_{i=1}^{n} L\big(y^{(i)}, f(x^{(i)})\big) \tag{1}$$

Se y for um valor contínuo, temos um problema de regressão. Se tivemos um valor discreto de y, trata-se de um problema de classificação. Um modelo de ML tem duas fases principais: treino e teste. Na fase de treino, o modelo aprende os parâmetros a partir dos dados que ele vê. Na fase de teste, ele avalia essa aprendizagem com dados que o modelo não viu, e o desempenho do modelo é avaliado nessa etapa.

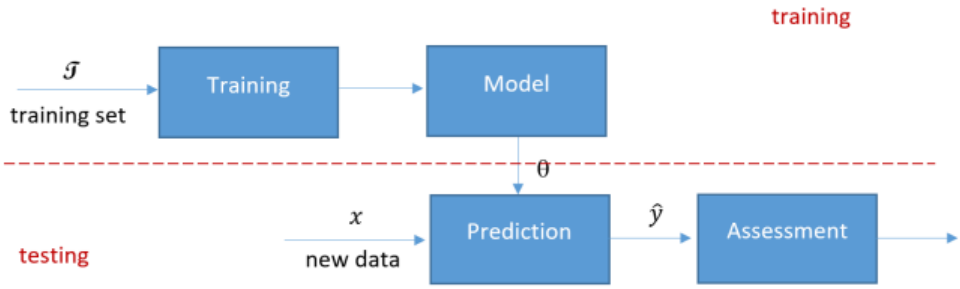

Figure 1: ML generic model system architecture.

Olhando para esta figura, surgem algumas perguntas: que classe de funções devemos considerar, como ajustamos essa função aos dados e como avaliamos o desempenho do modelo. Perguntas estas que vao ser respondidas ao longo da UC. Começando pelos métodos mais simples, temos o método do vizinho mais próximo (Nearest Neighbour, NN). Para prever um output y' a partir de um novo input x', vamos ao nosso conjunto de treino e procuramos o x mais próximo do novo x', atribuindo ao y' o valor y desse x. No que toca à generalização, é o método dos k vizinhos mais próximos (k-NN), onde, para a regressão, é calculado pela média e, para a classificação, é calculado pela classe que aparece mais vezes. Onde o parâmetro k é um hiperparâmetro, quando k é pequeno, temos maior variância e, quando k é grande, maior viés.

$$f(x) = \begin{cases} \text{mean}\big(y^{(1)}, \ldots, y^{(k)}\big), & \text{(regressão)} \\ \text{most voted class}\big(y^{(1)}, \ldots, y^{(k)}\big), & \text{(classificação)} \end{cases} \tag{2}$$

2. Linear Regression

Na regressão linear, o nosso principal objetivo é encontrar uma relação entre as variáveis independentes, os vários x, e a variável dependente contínua y. A ideia principal é encontrar uma reta que melhor ajuste todos os pontos (dados observados), pois assim obtém-se uma relação entre eles. Se estivermos a tentar prever o preço de um apartamento em Lisboa, conhecendo apenas a sua área, temos uma relação aproximadamente linear :

$$\hat{y} = f(x) = \beta_0 + \beta_1 x, \tag{3}$$

onde β_0 é o intercepto (valor previsto quando $x = 0$) e β_1 é a inclinação (quanto o preço aumenta por unidade de área). Quando temos um único ponto, há uma infinidade de retas que passam por ele. Quando temos dois pontos, existe apenas uma reta possível. Com 3 ou mais pontos, na maioria das vezes, não existe uma reta que passe por eles 3; portanto, temos que assumir que existe um erro de previsão (diferença entre os valores observados e os previstos). E temos que procurar a reta que minimiza exatamente esse erro de previsão (erro global). Cada erro é definido como:

$$e^{(i)} = y^{(i)} - \hat{y}^{(i)} = y^{(i)} - f\big(x^{(i)}\big), \tag{4}$$

e medimos a qualidade da previsão do modelo pela soma dos quadrados dos erros (SSE- Sum of Squared Errors):

$$\text{SSE} = \sum_{i=1}^{n} \big(y^{(i)} - f(x^{(i)})\big)^2. \tag{5}$$

E minimizar a expressão de SSE é o equivalente a encontrar a reta que minimiza o erro global. O problema de otimização em questão é:

$$\arg\min_{\beta_0,\beta_1} \sum_{i=1}^{n} \big(y^{(i)} - \beta_0 - \beta_1 x^{(i)}\big)^2. \tag{6}$$

Para encontrar o mínimo, calcula-se as derivadas parciais da expressão SSE, em ordem a β_0 e β_1 e iguala-se a zero:

$$\frac{\partial \text{SSE}}{\partial \beta_0} = 0, \qquad \frac{\partial \text{SSE}}{\partial \beta_1} = 0. \tag{7}$$

E, a partir disso, resultam as equações normais, um sistema linear em β_0 e β_1. Resolvem-se as equações e obtemos uma estimativa analítica dos parâmetros. Este método chama-se método dos mínimos quadrados (least squares). No caso que tenhamos várias variáveis, temos:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p. \tag{8}$$

Que em notação vetorial, fica:

$$\hat{y} = \begin{bmatrix} 1 & x_1 & \cdots & x_p \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix} = [1, \mathbf{x}^\top]\boldsymbol{\beta}. \tag{9}$$

Com um conjunto de treino de n amostras, construímos a nossa design matrix X (de dimensão $n \times (p+1)$), e o vetor de saídas $y \in \mathbb{R}^n$. O problema dos least squares escreve-se então como:

$$\arg\min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2. \tag{10}$$

Ao derivar em relação a $\boldsymbol{\beta}$ e igualar o gradiente a zero, obtemos:

$$\mathbf{X}^\top \mathbf{X}\, \hat{\boldsymbol{\beta}} = \mathbf{X}^\top \mathbf{y}. \tag{11}$$

Se a matriz $\mathbf{X}^\top\mathbf{X}$ for inversível (ou seja, não singular), a solução é:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}. \tag{12}$$

Esta é a famosa solução analítica da regressão linear. O modelo ajustado é então:

$$\hat{\mathbf{y}} = \mathbf{X}\,\hat{\boldsymbol{\beta}}. \tag{13}$$

A matrix $\mathbf{X}^\top\mathbf{X}$ é singular caso haja menos observações que variaveis, ou se existem features altamente correlacionadas. Nesta caso o sistema tem infinitas soluções e o modelo é instavel. A soma dos erros quadráticos (SSE) mede o erro absoluto do modelo, mas não é intuitiva porque depende da escala de y. Por isso, utiliza-se o coeficiente de determinação R^2, definido como:

$$R^2 = 1 - \frac{\mathrm{SS_{res}}}{\mathrm{SS_{tot}}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}. \tag{14}$$

Interpretando os valores de R^2:

$$\begin{aligned} R^2 = 1 \quad &\Rightarrow \text{ajuste perfeito (sem erro);} \\ R^2 = 0 \quad &\Rightarrow \text{o modelo não é melhor do que prever sempre a média;} \\ R^2 < 0 \quad &\Rightarrow \text{o modelo é pior do que a média.} \end{aligned}$$

O coeficiente R^2 é, portanto, uma métrica relativa que quantifica quanto da variabilidade dos dados é explicada pelo modelo linear. O **Teorema de Gauss–Markov** estabelece que, sob as hipóteses clássicas — ruído com média zero e variância constante ($\mathbf{w} \sim \mathcal{N}(0, \sigma^2\mathbf{I})$) —, o estimador de mínimos quadrados

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y} \tag{15}$$

é não enviesado (isto é, em média acerta no valor verdadeiro) e possui a menor variância possível entre todos os estimadores lineares não enviesados. Por essa razão, é denominado **BLUE** (*Best Linear Unbiased Estimator*). A distribuição do estimador $\hat{\boldsymbol{\beta}}$ é aproximadamente normal:

$$p(\hat{\boldsymbol{\beta}}) = \mathcal{N}\big(\boldsymbol{\beta},\, \sigma^2(\mathbf{X}^\top\mathbf{X})^{-1}\big). \tag{16}$$

Logo, o erro de previsão num ponto $\mathbf{x}_0$ é maior quando:

- o ruído dos dados (σ^2) é elevado;
- o ponto $\mathbf{x}_0$ está longe da média das *features* (extrapolação);
- as *features* são quase redundantes, tornando $\mathbf{X}^\top\mathbf{X}$ mal condicionado.

Existem casos onde existem várias variáveis dependentes (vários y), cada uma pode ser modelada separadamente:

$$\mathbf{y}_k = \mathbf{X}\boldsymbol{\beta}_k + \mathbf{w}_k, \qquad k = 1, \ldots, K. \tag{17}$$

Na forma matricial:

$$\mathbf{Y} = \mathbf{X}\mathbf{B} + \mathbf{W}. \tag{18}$$

Onde $\mathbf{Y}$ contém todas as saídas e $\mathbf{B}$ contém todas as colunas de coeficientes. A minimização do erro total resulta em:

$$\hat{\mathbf{B}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{Y}. \tag{19}$$

Muitos fenômenos observados no mundo real não são lineares. Uma alternativa é utilizar o modelo polinomial:

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_p x^p. \tag{20}$$

Embora o modelo seja não linear em x, continua a ser linear nos parâmetros β, logo é possível utilizar o método dos mínimos quadrados. No entanto, aumentar excessivamente p torna o modelo instável e conduz ao overfitting, em que o modelo se ajusta excessivamente ao ruído. Outra forma de introduzir não linearidade é utilizar funções base locais, como as **Gaussian Radial Basis Functions (RBFs)**:

$$G_k(x) = e^{-\frac{\|x-c_k\|^2}{2\sigma^2}}, \tag{21}$$

onde c_k são os *centros* (obtidos, por exemplo, através de *k-means* ou uniformemente espaçados). O modelo resultante é dado por:

$$f(x) = \sum_{k=1}^{p} w_k \, G_k(x), \tag{22}$$

em que os pesos w_k são estimados por mínimos quadrados. O conceito de *kernel* generaliza a ideia de semelhança entre pontos:

$$k(x, x') = \begin{cases} x^\top x', & \text{(linear)} \\ (1 + x^\top x')^d, & \text{(polinomial)} \\ e^{-\frac{\|x-x'\|^2}{2\sigma^2}}, & \text{(gaussiano)} \end{cases} \tag{23}$$

Os *kernels* permitem aplicar métodos lineares num espaço de alta dimensão sem o calcular explicitamente — técnica conhecida como **kernel trick**.

Assim, obtém-se a **regressão por kernel (kernel regression)**:

$$\hat{y}(x) = \sum_{j=1}^{n} \hat{\beta}_j \, k(x, x_j), \tag{24}$$

onde

$$\hat{\boldsymbol{\beta}} = \mathbf{K}^{-1}\mathbf{y}, \tag{25}$$

e $\mathbf{K}$ é a **matriz de Gram**, definida por:

$$K_{ij} = k(x_i, x_j). \tag{26}$$

Na prática, como $\mathbf{K}$ pode ser quase singular, utiliza-se uma forma **regularizada** — a **Kernel Ridge Regression**, tratada no próximo capítulo.

3. Regularization

3.1. Motivação

A regressão linear clássica estima os coeficientes β minimizando o erro quadrático:

$$\min_{\beta} \|y - X\beta\|^2. \tag{27}$$

No entanto, esta abordagem apresenta dois problemas frequentes:

- **Instabilidade numérica:** se as colunas de X forem correlacionadas ou quase redundantes, a matriz $X^\top X$ torna-se quase singular e a solução $\hat{\beta} = (X^\top X)^{-1} X^\top y$ fica extremamente sensível a pequenas variações nos dados;
- **Sobreajuste (overfitting):** quando o número de variáveis é elevado ou os dados contêm ruído, o modelo tende a ajustar-se excessivamente ao conjunto de treino, perdendo capacidade de generalização.

A *regularização* resolve ambos os problemas ao adicionar um termo de penalização da complexidade do modelo.

3.2. Intuição geral

Em vez de apenas minimizar o erro no treino, pretende-se controlar também a magnitude dos coeficientes. O novo problema passa a ser:

$$\min_{\beta} \underbrace{\|y - X\beta\|^2}_{\text{erro de ajuste}} + \lambda \underbrace{R(\beta)}_{\text{termo de regularização}}, \tag{28}$$

onde $\lambda > 0$ controla o equilíbrio entre os dois termos. Valores grandes de λ reduzem a complexidade do modelo, enquanto valores pequenos o tornam semelhante à solução de mínimos quadrados.

3.3. Ridge Regression (Regularização L2)

A forma mais simples de regularização é o **Ridge Regression**, que penaliza o quadrado da norma Euclidiana dos coeficientes:

$$\min_{\beta} \|y - X\beta\|^2 + \lambda \|\beta\|^2. \tag{29}$$

O termo $\|\beta\|^2 = \sum_j \beta_j^2$ penaliza coeficientes grandes, suavizando o modelo. O gradiente do custo é dado por:

$$\nabla_\beta E_{\text{Ridge}} = -2X^\top y + 2(X^\top X + \lambda I)\beta. \tag{30}$$

Igualando a zero obtém-se as *equações normais regularizadas*:

$$(X^\top X + \lambda I)\hat{\beta}_{\text{ridge}} = X^\top y, \tag{31}$$

e, portanto, a solução fechada:

$$\boxed{\hat{\beta}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top y.} \tag{32}$$

O termo λI assegura que a matriz é sempre inversível, mesmo que $X^\top X$ seja singular.

3.4. Interpretação geométrica e estatística

O problema pode ser formulado de forma equivalente como:

$$\min_{\beta} \|y - X\beta\|^2 \quad \text{sujeito a} \quad \|\beta\|^2 \leq \tau, \tag{33}$$

onde τ define o limite de complexidade do modelo. No espaço dos parâmetros, o termo de regularização define uma região esférica (bola L2), e o ponto ótimo é onde a elipse do erro quadrático tangencia essa esfera.

Do ponto de vista Bayesiano, o *ridge regression* equivale a assumir um *prior* Gaussiano sobre os coeficientes:

$$\beta \sim \mathcal{N}(0, \sigma_\beta^2 I). \tag{34}$$

3.5. Propriedades espectrais

A matriz $X^\top X$ é simétrica e pode ser diagonalizada:

$$X^\top X = V \Lambda V^\top, \tag{35}$$

onde $\Lambda = \mathrm{diag}(\lambda_1, \lambda_2, \dots)$. A solução *ridge* altera os autovalores:

$$(X^\top X + \lambda I)^{-1} = V(\Lambda + \lambda I)^{-1} V^\top. \tag{36}$$

Logo, os autovetores mantêm-se, mas cada autovalor é "encolhido":

$$\tilde{\lambda}_i = \frac{1}{\lambda_i + \lambda}. \tag{37}$$

Isto significa que o *ridge regression* atua como um filtro de suavização, reduzindo a influência das direções mais instáveis.

3.6. Lasso Regression (Regularização L1)

A regularização L2 encolhe todos os coeficientes, mas raramente os torna exatamente nulos. O **Lasso** (*Least Absolute Shrinkage and Selection Operator*) introduz a norma L1:

$$\min_\beta \|y - X\beta\|^2 + \lambda \|\beta\|_1, \tag{38}$$

onde:

$$\|\beta\|_1 = \sum_j |\beta_j|. \tag{39}$$

A penalização L1 cria uma região de otimização em forma de losango (bola L1), que frequentemente intercepta os eixos, forçando alguns coeficientes a zero. Desta forma, o Lasso promove **esparsidade** e **seleção automática de variáveis**.

3.7. Comparação entre Ridge e Lasso

Característica	**Ridge (L2)**	**Lasso (L1)**
Penalização	$\sum \beta_j^2$	$\sum \lvert\beta_j\rvert$
Efeito	Encolhe coeficientes	Zera coeficientes irrelevantes
Forma geométrica	Esfera (bola L2)	Losango (bola L1)
Aplicação típica	Colinearidade	Seleção de variáveis
Solução fechada	Sim	Não (convexa, numérica)

Table 1: Comparação entre Ridge e Lasso Regression.

3.8. Relação com esparsidade e norma L0

Idealmente, desejaríamos minimizar diretamente o número de coeficientes não nulos:

$$\min_{\beta} \|y - X\beta\|^2 + \lambda\|\beta\|_0, \tag{40}$$

onde $\|\beta\|_0$ conta o número de coeficientes diferentes de zero. Contudo, este problema é combinatorial e intratável. O Lasso surge como uma aproximação convexa que preserva a esparsidade de forma eficiente.

3.9. Regularização e centragem dos dados

Caso os dados não estejam centrados, deve-se primeiro subtrair as médias:

$$x'^{(i)} = x^{(i)} - \bar{x}, \quad y'^{(i)} = y^{(i)} - \bar{y}. \tag{41}$$

Após o treino, o modelo original é reconstruído como:

$$\hat{\beta}_0 = \bar{y} - \bar{x}^\top \hat{\beta}. \tag{42}$$

Isto garante consistência entre o modelo ajustado e os dados originais.

3.10. Kernel Ridge Regression (KRR)

Usando um *kernel* $k(x_i, x_j)$, é possível aplicar regularização *ridge* no espaço de features induzidas. Neste caso, a solução é dada por:

$$\hat{\beta} = (K + \lambda I)^{-1} y, \tag{43}$$

onde K é a matriz Gram com $K_{ij} = k(x_i, x_j)$. A previsão para um novo ponto x é:

$$\hat{y}(x) = \sum_{i=1}^{n} \hat{\beta}_i k(x, x_i). \tag{44}$$

O *Kernel Ridge Regression* combina a flexibilidade dos modelos não lineares com a estabilidade do *ridge*.

3.11. Escalamento e normalização

Diferentemente do método LS, o *ridge* e o *lasso* não são invariantes à escala das variáveis. É, portanto, essencial normalizar as *features* (por exemplo, com média zero e variância unitária) antes de aplicar regularização. Sem esta etapa, a escolha do valor ótimo de λ torna-se enviesada.

3.12. Interpretação do hiperparâmetro λ

O parâmetro λ é um hiperparâmetro de regularização:

- $\lambda = 0$: modelo coincide com mínimos quadrados;
- $\lambda \to \infty$: coeficientes tendem a zero.

O valor ótimo de λ é geralmente determinado por validação cruzada, equilibrando o compromisso entre *bias* e *variance*.

3.13. Resumo

Ou seja, na regressão linear clássica existe a estimação dos coeficientes Beta fazendo a minimização do erro quadrático. Mas podem surgir dois problemas principais, a instabilidade numérica e o overfitting. A instabilidade acontece quando a matriz XT X torna-se quase singular, quando temos mais observações do que inputs, por exemplo. Relativamente ao overfitting, ocorre quando há demasiadas features e o modelo sobreajusta-se excessivamente ao set de treino, perdendo a sua capacidade de generalização. Por isso que a regularização existe: resolve ambos os problemas ao adicionar um termo de penalização. Passamos a ter o erro de ajuste mais um termo de regularização multiplicado por um lambda que controla o equilíbrio entre ambos os termos. Vamos falar de alguns tipos de regularização, entre eles o Ridge regression (Regularização L2) e a Lasso regression (Regularização L1). No ridge regression, o termo de penalização é o quadrado da norma euclidiana dos coeficientes. O tal módulo de beta ao quadrado. Este termo penaliza coeficientes grandes, o que reduz o risco de instabilidade numérica. Pegando no gradiente de custo e igualando a zero, temos uma fórmula semelhante à dos least squares, mas com um termo extra lambda*I, o que torna a matriz invertível. Geometricamente, a regularização ridge resolve o problema original com uma restrição no tamanho dos vetores de coeficientes. No espaço dos parâmetros, o termo de regularização (L2) define uma esfera. De um ponto de vista estatístico, a regularização ridge equivale a assumir um prior gaussiano. No quesito de propriedades espectrais, a solução ridge altera os valores próprios, isto é, os valores próprios são mantidos, mas cada um é 'encolhido'. Nesta regressão, todos os coeficientes são encolhidos, mas raramente são igualados a zero. Muitas vezes não queremos trabalhar com todas as features, mas apenas com algumas. Daí utilizamos a lasso regression. Na lasso regression, o termo L1, geometricamente, cria um losango que intercepta os eixos. O que força alguns coeficientes a zero. Então, o lasso não apenas regulariza, mas também faz uma seleção automática. Em teoria, o modelo mais simples seria considerar algumas entradas de beta como não nulas. No entanto, este problema é combinatorial e intratável. Portanto, o Lasso surge como uma aproximação convexa deste problema. Se as nossas variáveis de entrada e de saída não estiverem centradas, o primeiro passo é subtrair as médias. A regularização aplica-se apenas às variáveis centradas. Recordando que um kernel mede a semelhança entre pontos. É possível aplicar a regularização no espaço de features induzidas por kernels. O KRR combina a potência dos modelos não lineares (kernels) com a estabilidade do método do ridge. Ao contrário do método LS, o ridge e o lasso não são invariantes à escala das features. Portanto, é essencial normalizar as features antes de aplicar a regularização.

Método	**Formulação**	**Efeito principal**
Ridge Regression	$\min_\beta \|y - X\beta\|^2 + \lambda\|\beta\|^2$	Reduz variância, suaviza
Lasso Regression	$\min_\beta \|y - X\beta\|^2 + \lambda\|\beta\|_1$	Gera esparsidade, seleciona variáveis
L0 Regularization (teórica)	$\min_\beta \|y - X\beta\|^2 + \lambda\|\beta\|_0$	Seleção ideal, intratável
Kernel Ridge Regression	$\hat{\beta} = (K + \lambda I)^{-1} y$	Combina kernels com estabilidade

Table 2: Resumo dos principais métodos de regularização.

4. Evaluation and Generalization

4.1. Por que precisamos de avaliar?

Treinar um modelo é apenas metade do processo. O verdadeiro objetivo de um algoritmo de *Machine Learning* não é ajustar-se perfeitamente aos dados de treino, mas sim **generalizar**, isto é, apresentar bom desempenho em dados nunca antes vistos. A avaliação serve, portanto, para medir quão bem o modelo aprendeu padrões genuínos, em vez de apenas memorizar o conjunto de treino.

4.2. Dados de treino, validação e teste

O conjunto de dados é geralmente dividido em três subconjuntos:

- **Treino (training set):** utilizado para ajustar os parâmetros do modelo (β, pesos, etc.);
- **Validação (validation set):** usado para ajustar hiperparâmetros (por exemplo, o valor de λ na regularização);
- **Teste (test set):** usado apenas no final para avaliar a performance real do modelo.

Se usássemos o mesmo conjunto para treino e teste, o modelo poderia simplesmente "decorar" os dados, levando a uma falsa impressão de bom desempenho.

4.3. Função de risco

O desempenho teórico de um modelo é medido através da **função de risco esperado**:

$$R(f) = \mathbb{E}_{x,y}[L(y, f(x))], \tag{45}$$

onde $L(y, f(x))$ representa a função de perda (por exemplo, o erro quadrático). Como a distribuição verdadeira dos dados é desconhecida, estima-se o risco através do **risco empírico**:

$$R_{emp}(f) = \frac{1}{n} \sum_{i=1}^{n} L(y_i, f(x_i)). \tag{46}$$

O objetivo é encontrar um modelo com baixo risco verdadeiro, e não apenas baixo risco empírico.

4.4. Overfitting

O *overfitting* ocorre quando o modelo aprende demasiado os detalhes do treino, incluindo o ruído. Tipicamente:

- o erro de treino continua a diminuir com a complexidade do modelo;
- o erro de teste começa a aumentar após certo ponto.

Esse ponto representa o *limite ótimo de complexidade*, onde o modelo equilibra *bias* e *variance*.

4.5. Trade-off Bias–Variance

O erro quadrático esperado pode ser decomposto em três componentes:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}. \tag{47}$$

- **Bias:** erro sistemático devido à simplificação do modelo (modelo demasiado simples);
- **Variance:** sensibilidade do modelo a pequenas variações nos dados (modelo demasiado complexo);
- **Irreducible Error:** ruído inerente aos dados, impossível de eliminar.

A regularização e a escolha adequada do modelo permitem controlar este equilíbrio.

4.6. Generalização e erro esperado

A diferença entre o erro de treino e o erro de teste define o **erro de generalização**. Modelos com elevado erro de generalização ajustaram-se demasiado aos dados de treino, enquanto bons modelos mantêm erros semelhantes nos dois conjuntos, indicando padrões estáveis e não aleatórios.

4.7. Métodos de avaliação

Os métodos de avaliação mais utilizados são:

- **Hold-out:** divisão simples (por exemplo, 80% treino / 20% teste). Rápido, mas dependente da aleatoriedade da divisão;
- **K-Fold Cross-Validation:** o conjunto é dividido em k blocos. O modelo é treinado k vezes, deixando um bloco diferente para teste em cada iteração. O erro médio fornece uma estimativa robusta do desempenho;
- **Leave-One-Out (LOOCV):** caso extremo da *cross-validation* com $k = n$, em que cada amostra é usada como teste uma vez. Mais exato, mas computacionalmente dispendioso.

A *cross-validation* é a técnica padrão para estimar o desempenho e selecionar hiperparâmetros de forma fiável.

4.8. Seleção de hiperparâmetros

Os **hiperparâmetros** (como λ, número de vizinhos, profundidade de árvores, etc.) não são aprendidos diretamente pelo modelo. São ajustados através de validação, comparando o desempenho de múltiplos modelos e escolhendo aquele que minimiza o erro médio de validação. Este processo é conhecido como *model selection* e é essencial para evitar o *overfitting*.

4.9. Cross-validation aninhada (Nested CV)

Quando o conjunto de dados é pequeno, usar o mesmo processo de validação para ajustar hiperparâmetros e avaliar desempenho pode causar viés. A solução é a **cross-validation aninhada**, composta por duas camadas:

- uma camada interna para seleção de hiperparâmetros;
- uma camada externa para avaliar o desempenho final.

Desta forma, o teste externo permanece totalmente independente do treino.

4.10. Métricas de avaliação

As métricas variam conforme o tipo de problema:

- **Regressão:** erro quadrático médio (MSE), erro absoluto médio (MAE), coeficiente de determinação R^2;
- **Classificação:** acurácia, precisão, revocação (*recall*), F1-score, curva ROC-AUC.

Em problemas desbalanceados, a acurácia pode ser enganadora; o F1-score fornece uma avaliação mais justa, equilibrando precisão e revocação.

4.11. Regularização e generalização

A regularização é uma ferramenta direta de controlo da generalização. Um λ maior aumenta o *bias* (modelo mais simples), mas reduz a *variance* (modelo mais estável). A escolha ótima de λ permite atingir o ponto ideal do *bias–variance trade-off*.

4.12. Influência do tamanho do dataset

Com mais dados, a variância tende a diminuir, tornando o modelo mais estável e menos dependente de regularização. Assim, um princípio fundamental em *Machine Learning* é: *"mais dados superam modelos mais complexos"*. A generalização melhora naturalmente à medida que a amostra se aproxima da distribuição real.

4.13. Learning Curves

As **learning curves** mostram o erro de treino e o erro de teste em função do número de exemplos usados:

- Erros altos nos dois conjuntos $\Rightarrow$ alto *bias* (subajuste);
- Erro de treino baixo e erro de teste alto $\Rightarrow$ *overfitting*;
- Erros convergentes e baixos $\Rightarrow$ bom equilíbrio.

Estas curvas são úteis para diagnosticar se o problema se deve à falta de dados ou à complexidade excessiva do modelo.

4.14. Conclusão

A avaliação de modelos não se resume a medir um número de erro, mas sim a compreender o equilíbrio entre **ajuste e generalização**. O objetivo final do *Machine Learning* é alcançar bom desempenho fora da amostra. Técnicas como *cross-validation*, regularização e *learning curves* são as ferramentas essenciais para atingir esse equilíbrio.

4.15. Resumo

Treinar um modelo é apenas uma parte do que se faz em ML. O principal objetivo é exatamente criar um modelo com boa capacidade de generalização, para que, quando aplicado a dados novos, produza outputs coerentes com a realidade. Daí, a necessidade de avaliar os modelos. Um conjunto de dados, normalmente, é dividido em 3 partes principais: treino, validação e teste (usado para a avaliação de desempenho do modelo). É necessário haver essa separação, pois, se o mesmo conjunto for utilizado para tudo, o modelo decora os resultados e obtemos desempenho irrealista. Matematicamente, o desempenho de um modelo é medido através da função de risco esperada, que, por sua

vez, é escrita em função da loss function. O overfitting acontece quando o modelo aprende demasiados detalhes do treino, incluindo o ruído. O desempenho de um modelo pode ser decomposto em 3 componentes: bias (erro sistematico devido à simplificação do modelo), variança (Sensibilidade do modelo a pequenas variações nos dados) e ruído (inerente aos dados). A diferença entre o erro no treino e o no teste é o erro de generalização. Os métodos de avaliação mais comuns são: Hold out (Divisão simples, p. ex., 80 % Treino e 20 % Teste), K-Fold Cross Validation (divide o conjunto em k partes) e Leave-One.Out (caso extremo de cross-validation com $k = n$). Os hiperparâmetros (p. ex., lambda da regularização, número de vizinhos no k-NN) não são aprendidos diretamente; precisam ser ajustados. A ideia é testar vários valores destes parâmetros e escolher os que minimizam o erro médio de validação. No que toca às métricas de avaliação, na regressão, temos o MSE, o MAE e o R^2. Na classificação temos acurácia , precisão, recall, F1-Score, etc. A regularização é uma ferramenta direta de controlo da generalização. Com o aumento do dataset, a variância tende a diminuir e o modelo fica mais estável e menos dependente da regularização.

Conceito	**Ideia central**	**Exemplo**
Overfitting	Modelo demasiado complexo, erro baixo no treino e alto no teste	Polinómio de alta ordem
Bias–Variance Trade-off	Equilíbrio entre simplificação e sensibilidade	Ridge Regression controla a variância
Cross-Validation	Divisão em múltiplos blocos para avaliação robusta	5-Fold, 10-Fold
Hiperparâmetros	Ajustados por validação, não aprendidos	λ, profundidade de árvore
Métricas	Dependem do tipo de problema	MSE, F1-score, ROC-AUC
Learning Curves	Diagnóstico visual de *overfitting/underfitting*	Erros de treino e teste

Table 3: Resumo dos principais conceitos de avaliação e generalização.

5. Optimization

5.1. Objetivo da Otimização

Em todos os modelos de *Machine Learning*, existe uma função de perda que mede o erro entre as previsões do modelo e os valores reais. O objetivo da otimização é encontrar o conjunto de parâmetros que minimiza essa função de perda:

$$\hat{\theta} = \arg\min_{\theta} L(\theta), \tag{48}$$

onde $L(\theta)$ representa a *loss function* (por exemplo, erro quadrático, entropia cruzada) e θ denota os parâmetros do modelo (como pesos ou coeficientes). A forma como procuramos este mínimo — analítica ou numérica — define o método de otimização.

5.2. Tipos de Problemas de Otimização

Podemos distinguir dois grandes tipos:

- **Otimização convexa:** a função de perda possui um único mínimo global (ex.: regressão linear, ridge regression);
- **Otimização não convexa:** existem múltiplos mínimos locais (ex.: redes neuronais profundas).

Num problema convexo, qualquer ponto onde o gradiente é nulo é o mínimo global. Em funções não convexas, o gradiente pode anular-se em mínimos locais ou em pontos de sela, exigindo estratégias mais robustas.

5.3. Soluções Analíticas e Numéricas

Alguns problemas, como a regressão linear, admitem uma solução analítica fechada:

$$\hat{\beta} = (X^\top X)^{-1} X^\top y. \tag{49}$$

Em casos mais complexos (como redes neuronais), não existe solução explícita, sendo necessário recorrer a métodos numéricos iterativos que convergem para um mínimo aproximado.

5.4. Gradiente Descendente

O **Gradient Descent (GD)** é o método mais comum para otimizar funções em *Machine Learning*. A ideia é começar com um valor inicial θ_0, calcular o gradiente da função de perda e dar um passo na direção oposta ao gradiente:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t), \tag{50}$$

onde η é a **learning rate**, que controla o tamanho do passo. Um valor demasiado grande pode fazer o algoritmo divergir; um valor demasiado pequeno torna-o lento.

5.5. Intuição Geométrica

A função de perda pode ser vista como uma "paisagem" de montanhas e vales. O gradiente indica a direção de maior subida, e o gradiente descendente move-se na direção oposta — a de descida mais íngreme. Se a função for convexa, o processo converge para o mínimo global; se for não convexa, pode ficar preso em mínimos locais.

5.6. Cálculo do Gradiente

Para funções multivariadas, o gradiente é o vetor das derivadas parciais:

$$\nabla_\theta L(\theta) = \begin{bmatrix} \frac{\partial L}{\partial \theta_1} \\ \frac{\partial L}{\partial \theta_2} \\ \vdots \\ \frac{\partial L}{\partial \theta_p} \end{bmatrix}. \tag{51}$$

Este vetor aponta na direção de crescimento mais rápido da função, enquanto o seu oposto $-\nabla_\theta L$ aponta na direção de descida.

5.7. Variações do Gradiente Descendente

Existem várias implementações do GD, dependendo da forma como os dados são utilizados:

- **Batch Gradient Descent:** calcula o gradiente usando todas as amostras de treino. É estável, mas computacionalmente pesado;
- **Stochastic Gradient Descent (SGD):** usa apenas uma amostra por iteração. É rápido, mas mais ruidoso;
- **Mini-Batch Gradient Descent:** utiliza pequenos subconjuntos (*batches*) de dados, equilibrando velocidade e estabilidade. É o método mais comum atualmente.

5.8. Escolha da Learning Rate

A *learning rate* η é um hiperparâmetro crítico:

- η muito alta $\Rightarrow$ o algoritmo pode oscilar ou divergir;
- η muito baixa $\Rightarrow$ convergência lenta.

Métodos modernos utilizam taxas de aprendizagem adaptativas que se ajustam dinamicamente ao longo do treino.

5.9. Método do Momentum

O **Momentum** é uma extensão do GD que acumula parte do gradiente das iterações anteriores, suavizando o percurso da descida. A atualização é feita por:

$$v_{t+1} = \gamma v_t + \eta \nabla_\theta L(\theta_t), \quad \theta_{t+1} = \theta_t - v_{t+1}, \tag{52}$$

onde $\gamma \in [0, 1]$ é o fator de memória. O *momentum* ajuda a ultrapassar mínimos rasos e acelera a convergência em direções consistentes.

5.10. Métodos Adaptativos

Métodos adaptativos ajustam automaticamente a taxa de aprendizagem para cada parâmetro. Entre os mais usados estão:

- **Adagrad:** adapta η com base na magnitude dos gradientes anteriores, reduzindo passos para parâmetros com grandes gradientes;
- **RMSProp:** mantém uma média móvel dos quadrados dos gradientes para evitar que η diminua excessivamente;
- **Adam (Adaptive Moment Estimation):** combina *momentum* e *RMSProp*, ajustando as taxas de aprendizagem de forma eficiente e estável.

O Adam é atualmente o método padrão em *Deep Learning*.

5.11. Critérios de Paragem

Os métodos iterativos requerem condições de paragem. Os critérios mais comuns são:

- Gradiente pequeno: $\|\nabla_\theta L(\theta_t)\| < \varepsilon$;
- Erro de validação deixa de melhorar (*early stopping*);
- Número máximo de iterações atingido.

O *early stopping* previne o *overfitting*, interrompendo o treino quando o erro de validação começa a aumentar.

5.12. Problemas Práticos

Mesmo com bons algoritmos, surgem dificuldades como:

- **Múltiplos mínimos locais:** comuns em funções não convexas;
- **Pontos de sela:** regiões onde o gradiente é nulo mas a função não tem mínimo;
- **Escalas desiguais:** *features* com magnitudes muito diferentes fazem o gradiente oscilar, tornando essencial a normalização dos dados.

Estas dificuldades justificam o uso de técnicas como regularização, normalização e inicialização adequada dos pesos.

5.13. Otimização e Aprendizagem

A otimização é o mecanismo central do processo de aprendizagem:

- O gradiente descendente ajusta os parâmetros para minimizar o erro;
- A regularização impede passos excessivos e controla a complexidade;
- A avaliação verifica se o processo resultou em boa generalização.

Assim, otimizar é, em essência, o ato de **aprender a partir dos dados**.

5.14. Resumo

Em todos os modelos de ML, existe uma função que mede o erro entre o que foi observado e o que foi previsto. O principal objetivo da otimização é encontrar o conjunto de parâmetros que minimize essa loss function. Existem dois tipos de otimização: a convexa e a não convexa. Na convexa, há apenas um mínimo global (como na regressão); na não convexa, há vários mínimos locais (como em redes neuronais). Na regressão linear simples, podemos obter uma solução analítica fechada. Mas nas redes neurais profundas, por exemplo, não há solução explícita e é aí que entram os métodos numéricos iterativos que convergem para um ponto mínimo aproximado. O gradient descent (GD) é o método mais utilizado para otimizar funções em ML. Começamos num ponto inicial; calcula-se o gradiente da loss function e vai-se no sentido oposto (pois queremos minimizar esta função). Para controlar o tamanho do passo, utiliza-se um parâmetro chamado learning rate. Para funções com muitas variáveis, o gradiente é o vetor das suas respetivas derivadas parciais. Existem algumas maneiras de implementar o GD: batch gradient descent (calcula o gradiente com todos os dados de treino de uma só vez), stochastic gradient descent (usa apenas uma amostra de cada vez) e mini-batch gradient descent (equilibrio entre os dois). O learning rate é um hiperparâmetro muito importante. Se for um valor muito elevado, o algoritmo pode saltar o mínimo e oscilar; se for muito baixo, o processo é lento e pode parar antes de atingir o mínimo. O método do momentum é uma melhoria sobre o GD básico. A principal ideia é acumular parte do gradiente das iterações anteriores, o que suaviza a descida. Para evitar ajustar manualmente o learning rate, surgiram os métodos adaptativos: o Adagrad (adapta a lr com base na magnitude dos gradientes anteriores), o RMSProp (mantém uma média móvel dos quadrados dos gradientes) e o Adam (usa medias móveis tanto dos gradientes quanto dos seus quadrados). Os algoritmos iterativos precisam de uma condição para parar; podem ser: quando o gradiente torna-se inferior a um parâmetro epsilon, quando o erro de validação deixa de melhorar (early stopping) e quando o número máximo de iterações é atingido. Mesmo com bons métodos, existem sempre desafios a serem contornados, como o facto de haver múltiplos mínimos locais, pontos de sela e escalonamento desigual.

Conceito	Ideia Central	Exemplo / Observação
Função de perda	Mede o erro do modelo	Erro quadrático, entropia cruzada
Gradiente descendente	Atualiza parâmetros na direção de descida	$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$
Learning rate (η)	Controla o tamanho dos passos	Pequena: lenta; Grande: instável
Momentum	Acumula gradientes anteriores	Acelera convergência
Adam	Método adaptativo padrão	Combina momentum e RMSProp
Critérios de paragem	Define fim do treino	Gradiente pequeno, early stopping
Problemas práticos	Múltiplos mínimos, escalas desiguais	Normalização e regularização ajudam

Table 4: Resumo dos principais conceitos de otimização.

6. Neural Networks

6.1. Motivação

Os modelos lineares assumem que a relação entre as variáveis de entrada e de saída é linear. No entanto, muitos fenómenos reais são não lineares — como reconhecimento de imagens, voz ou padrões complexos. As **redes neuronais artificiais** foram criadas para capturar este tipo de relação, através da composição de várias transformações não lineares. O princípio base é empilhar camadas de neurónios, onde cada camada aprende uma representação mais abstrata dos dados.

6.2. O neurónio artificial

Cada neurónio recebe um vetor de entradas $x = [x_1, x_2, \ldots, x_p]$, aplica um conjunto de pesos $w = [w_1, w_2, \ldots, w_p]$ e um termo de *bias* b, produzindo:

$$z = w^\top x + b. \tag{53}$$

Depois, aplica-se uma função de ativação $g(\cdot)$, introduzindo não linearidade:

$$a = g(z). \tag{54}$$

O valor a é propagado para as camadas seguintes. Sem a função de ativação, mesmo várias camadas equivaleriam a uma única transformação linear.

6.3. Funções de ativação

As funções de ativação determinam o tipo de não linearidade introduzida. As mais comuns são:

- **Sigmóide:** $g(z) = \frac{1}{1+e^{-z}}$ (saída entre 0 e 1);
- **Tanh:** $g(z) = \tanh(z)$ (centrada em 0, simétrica);
- **ReLU (Rectified Linear Unit):** $g(z) = \max(0, z)$ (atualmente a mais usada);

- **Leaky ReLU / ELU:** variações da ReLU que evitam o "apagamento" de neurónios.

A escolha da função de ativação afeta diretamente o desempenho e a estabilidade do treino.

6.4. Arquitetura das redes neuronais

Uma rede neuronal é composta por:

- **Camada de entrada:** recebe as *features* originais;
- **Camadas escondidas (hidden layers):** realizam transformações intermédias;
- **Camada de saída:** gera a previsão final.

O número de camadas e de neurónios por camada determina a **capacidade de representação** da rede. Redes com uma só camada escondida podem aproximar qualquer função contínua (Teorema da Aproximação Universal), mas redes mais profundas fazem-no de forma mais eficiente.

6.5. Forward propagation

Durante o treino, os dados percorrem a rede no sentido direto (*forward pass*):

$$a^{[l]} = g(W^{[l]}a^{[l-1]} + b^{[l]}), \tag{55}$$

onde $a^{[l-1]}$ são as ativações da camada anterior, $W^{[l]}$ os pesos, e $b^{[l]}$ o *bias* da camada l. A última camada gera a previsão $\hat{y}$.

6.6. Função de perda

O erro entre as previsões e os valores reais é medido por uma função de perda $L(\hat{y}, y)$, que depende do tipo de problema:

- Regressão $\rightarrow$ Erro Quadrático Médio (MSE);
- Classificação $\rightarrow$ Entropia Cruzada (Cross-Entropy Loss).

O objetivo do treino é minimizar esta função em relação a todos os pesos e *biases*.

6.7. Backpropagation

O algoritmo de **backpropagation** calcula as derivadas parciais da função de perda em relação a todos os parâmetros da rede, aplicando a regra da cadeia. O processo ocorre em duas fases:

1. **Forward pass:** cálculo das ativações e da perda $L(\hat{y}, y)$;
2. **Backward pass:** propagação do erro de volta pelas camadas, obtendo $\frac{\partial L}{\partial W^{[l]}}$ e $\frac{\partial L}{\partial b^{[l]}}$.

Os gradientes obtidos são utilizados pelo otimizador (ex.: GD ou Adam) para atualizar os parâmetros. O *backpropagation* é, portanto, a aplicação sistemática da regra da cadeia em redes compostas.

6.8. Importância da não linearidade

Sem funções de ativação não lineares, uma rede de várias camadas seria equivalente a uma simples transformação linear:

$$W^{[3]}W^{[2]}W^{[1]}x = W_{\text{total}}x. \tag{56}$$

Logo, as funções de ativação são essenciais para que a rede aprenda fronteiras de decisão complexas.

6.9. Regularização em redes neuronais

Redes com muitos parâmetros estão sujeitas a *overfitting*. As principais estratégias para mitigá-lo são:

- **Weight decay (L2):** penaliza pesos grandes;
- **Dropout:** desativa aleatoriamente neurónios durante o treino;
- **Early stopping:** interrompe o treino quando o erro de validação começa a aumentar.

Estas técnicas reduzem a variância e aumentam a capacidade de generalização.

6.10. Inicialização dos pesos

A inicialização correta dos pesos é crucial para a estabilidade do treino. Valores muito pequenos ou muito grandes podem causar gradientes nulos ou explosivos. As estratégias mais comuns incluem:

- **Xavier Initialization:** adequada para funções sigmóide/tanh;
- **He Initialization:** ideal para redes com ReLU.

Estas técnicas mantêm a variância das ativações aproximadamente constante entre camadas.

6.11. Vanishing e Exploding Gradients

Em redes profundas, o gradiente pode:

- **Desaparecer (vanishing):** torna-se quase zero, impedindo o treino das camadas iniciais;
- **Explodir (exploding):** cresce exponencialmente, causando instabilidade numérica.

Estes problemas são mitigados com inicializações adequadas, normalização e funções de ativação como a ReLU.

6.12. Batch Normalization

A **Batch Normalization** normaliza as ativações de cada camada, impondo média 0 e variância 1 em cada mini-batch:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \tag{57}$$

onde μ_B e σ_B^2 são a média e variância do batch. Depois, aplica-se uma transformação linear aprendida:

$$y = \gamma \hat{x} + \beta. \tag{58}$$

Esta técnica estabiliza o treino e tem efeito regularizador.

6.13. Aprendizagem hierárquica

Em redes profundas, cada camada aprende representações progressivamente mais abstratas:

- As primeiras camadas captam padrões simples (bordas, cores);
- As camadas intermédias aprendem combinações mais complexas;
- As últimas camadas aprendem conceitos de alto nível (formas, objetos, semântica).

Esta hierarquia de representações é a base do sucesso das redes profundas em tarefas complexas.

6.14. Resumo

Os modelos lineares assumem que a relação entre as variáveis de entrada e de saída é linear. Mas muito do que é verificado no mundo real, tem comportamentos não-lineares. As redes neuronais foram criadas exatamente para capturar estas relações mais complexas. A ideia é criar um modelo composto por várias camadas, permitindo que o sistema aprenda representações de maior complexidade.

As redes neuronais são inspiradas na biologia e compostas por unidades chamadas de neurônios artificiais. Cada neurônio recebe um vetor de inputs, aplica um conjunto de pesos e um termo de bias e produz uma saída. Depois é aplicada uma função de ativação para introduzir não-linearidade. E este resultado é transmitido aos neurônios seguintes. Existem inúmeras funções de ativação, sendo as mais comuns: sigmoid (muito utilizada em classificadores binários), ReLU e tanh.

A arquitetura das redes neuronais baseia-se em camada de entrada (recebe as features), camadas escondidas (onde são realizadas as transformações) e camada de saída (resultado final, regressão ou classificação). Durante o treino, os dados percorrem a rede no sentido direto (forward propagation). O erro entre as previsões e os valores reais é medido por meio da função de perda, que depende do tipo de problema. Para ajustar os pesos da rede, utiliza-se o algoritmo de backpropagation, que aplica a regra da cadeia para calcular as derivadas parciais da função de perda. Este processo tem duas fases principais, a forward pass (calcula o output da rede e a perda da loss function) e a backward pass (propaga o erro da saída até às camadas iniciais).

Sem as funções de ativação não lineares, uma rede de múltiplas camadas acabaria em uma única transformação linear. Redes com muitos parâmetros são propensas ao overfitting, por isso usamos o weight decay (L2 Reg, penaliza pesos grandes), dropout (desativa aleatoriamente neurônios durante o treino) e early stopping. A forma como os pesos são inicializados afeta a estabilidade do treino; se forem inicializados com valores demasiado pequenos ou grandes, podem levar a gradientes nulos ou explosivos. Para isso, temos estratégias comuns como o Xavier Initialization e o He Initialization.

Em redes muito profundas, o gradiente pode desaparecer (tendendo para zero) ou explodir (crescendo exponencialmente). A batch normalization é uma técnica que normaliza as ativações em cada camada, forçando-as a ter média 0 e variância 1 em cada mini-batch. Isto acelera e estabiliza o treino, além de ter um efeito regularizador.

7. Data Classification

A **classificação** é uma das tarefas fundamentais em Aprendizagem Automática. O objetivo é atribuir uma classe $y \in \Omega$ (onde $\Omega = \{\omega_0, \ldots, \omega_{K-1}\}$) a uma nova observação $x \in \mathbb{R}^p$, com base num conjunto de dados de treino. O mapeamento $f(x) : \mathbb{R}^p \to \Omega$ é denominado **classificador**, e o processo de aprendizagem consiste em ajustar f de modo a generalizar bem para novos exemplos.

7.1. Discriminant Functions

Uma forma alternativa de definir um classificador é através de um conjunto de **funções discriminantes** $f_i(x)$, uma para cada classe. A amostra x é atribuída à classe ω_i se

$$f_i(x) \geq f_j(x), \quad \forall j \neq i.$$

Estas funções transformam o problema de classificação num problema de comparação de valores reais. Quando duas funções assumem o mesmo valor, a decisão é ambígua.

As fronteiras entre classes — denominadas **decision boundaries** — correspondem aos pontos onde duas funções discriminantes são iguais, dividindo o espaço de entrada em regiões de decisão $R_i = \{x : f(x) = \omega_i\}$. Cada região representa o domínio de uma classe, e o conjunto de todas as regiões forma uma partição do espaço.

7.2. Classifier Design

O design de um classificador depende da informação disponível:

- No **caso ideal**, conhece-se a distribuição conjunta $p(x, y)$;
- No **caso prático**, apenas se dispõe de um conjunto de treino $T = \{(x^{(i)}, y^{(i)})\}$.

Quando as distribuições são conhecidas, é possível construir um classificador ótimo com base em critérios probabilísticos — este é o **classificador de Bayes**.

7.3. Bayes Classifier (Binary Loss)

O **classificador de Bayes** minimiza o risco esperado (ou custo médio de erro). Se considerarmos uma **função de perda binária**, que atribui 0 a decisões corretas e 1 a incorretas, o risco torna-se a probabilidade de erro de classificação. Assim, o classificador ótimo associa x à classe com maior probabilidade a posteriori:

$$f(x) = \arg\max_{\omega \in \Omega} P(\omega|x).$$

Esta regra é conhecida como **regra de decisão de Bayes**, e estabelece o limite teórico de desempenho: nenhum classificador pode ter erro médio menor do que o de Bayes, assumindo conhecimento total da distribuição dos dados.

7.4. Bayes Risk and General Loss Functions

Em situações mais realistas, diferentes erros têm diferentes custos (por exemplo, confundir "sem tumor" com "com tumor" é mais grave do que o inverso). Neste caso, define-se uma **matriz de perdas** L_{ij}, onde $L_{ii} = 0$ e $L_{ij} > 0$ para $i \neq j$. O **risco esperado** é então dado por:

$$R = \mathbb{E}\{L(y, \hat{y}(x))\} = \sum_y \int L(y, \hat{y}(x))\, p(x, y)\, dx.$$

Para minimizar este risco, calcula-se o custo esperado de escolher uma classe ω dado x:

$$c_\omega(x) = \sum_{y \in \Omega} L(y, \omega)\, P(y|x),$$

e escolhe-se a classe com menor custo esperado:

$$f(x) = \arg\min_{\omega \in \Omega} c_\omega(x).$$

Quando a perda é binária, este modelo reduz-se novamente à regra de Bayes.

7.5. Intuição e Exemplos

Num cenário médico, considerar a perda uniforme

$$L = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

significa tratar todos os erros de forma igual. No entanto, uma matriz assimétrica, como

$$L = \begin{bmatrix} 0 & 1 \\ 5 & 0 \end{bmatrix},$$

penaliza mais os *falsos negativos* (classificar um tumor como saudável). Assim, a decisão ótima pode mudar — o modelo tenderá a classificar mais casos como "tumor" para reduzir o risco total. Este formalismo mostra como o **conceito de risco esperado** generaliza a probabilidade de erro e permite incorporar informação sobre os custos das decisões.

7.6. Summary of Main Concepts

A classificação é uma das tarefas fundamentais em Aprendizagem Automática. O seu objetivo é atribuir uma classe $y \in \Omega$ a uma nova observação $x \in \mathbb{R}^p$, com base num conjunto de treino previamente rotulado. Uma forma equivalente de representar um classificador é através de um conjunto de funções discriminantes $f_i(x)$, cada uma associada a uma classe ω_i. A amostra x é atribuída à classe cuja função discriminante apresenta o maior valor. Quando duas funções discriminantes são iguais, a decisão é ambígua. As fronteiras entre classes, designadas por *decision boundaries*, correspondem aos pontos em que as funções discriminantes de classes distintas têm o mesmo valor. Cada função define uma região de decisão correspondente à sua classe. O design de um classificador depende da informação disponível. No caso ideal, conhece-se a distribuição conjunta dos dados $p(x, y)$; no caso prático, apenas se dispõe de um conjunto de treino. Quando a distribuição é conhecida, é possível construir um classificador ótimo com base em critérios estatísticos — o **classificador de Bayes**. Este classificador minimiza o

risco esperado, isto é, o valor médio da função de perda ponderado pela probabilidade de ocorrência de cada par (x, y). Em contextos mais realistas, diferentes tipos de erro podem ter custos distintos. Nesse caso, introduz-se uma **matriz de perdas** L_{ij} que define o custo associado à decisão de classificar um exemplo da classe ω_i como pertencente à classe ω_j.

Concept	**Main Idea**	**Connection**
Classifier $f(x)$	Maps observations to classes	Ultimate goal of the classification task
Discriminant functions $f_i(x)$	Compare classes numerically	Define decision boundaries
Decision regions	Partition input space by $f(x)$	Define class domains
Expected risk R	Average loss weighted by cost	Optimization criterion
Loss function $L(y, \hat{y})$	Defines penalties for errors	Shapes optimal decision
Bayes rule	Chooses $\max P(\omega\|x)$	Special case of risk minimization
Generalized loss	Allows asymmetric costs	Crucial in sensitive domains (e.g. medicine)

Table 5: Main concepts of Data Classification.

8. Linear Classifiers

Os classificadores lineares são modelos que separam as classes através de fronteiras de decisão lineares no espaço das variáveis de entrada. O objetivo é determinar uma função de decisão que atribua cada observação $x \in \mathbb{R}^p$ à classe correta, de forma simples e eficiente. Estes métodos são amplamente utilizados pela sua interpretabilidade e baixo custo computacional, embora assumam que as classes possam ser separadas aproximadamente por planos lineares.

8.1. Definição Geral

Um classificador linear define, para cada classe ω_i, uma função discriminante linear ou afim:

$$f_i(x) = \beta_{0i} + \beta_i^T x$$

A amostra x é atribuída à classe cuja função discriminante apresenta o maior valor:

$$f(x) = \arg\max_i f_i(x)$$

A fronteira de decisão entre duas classes ω_i e ω_j é o conjunto de pontos onde $f_i(x) = f_j(x)$; esta fronteira é um **hiperplano linear** no espaço das features.

8.2. Linear Discriminant Analysis (LDA)

O **LDA** é um modelo probabilístico baseado na regra de Bayes. Assume que as distribuições condicionais de cada classe são gaussianas, com a mesma matriz de covariância:

$$p(x|\omega_k) = \mathcal{N}(\mu_k, \Sigma)$$

As probabilidades a priori $P(\omega_k)$ e as médias μ_k são estimadas a partir dos dados de treino. A probabilidade a posteriori é dada por:

$$P(\omega_k|x) \propto P(\omega_k)\, p(x|\omega_k)$$

Tomando o logaritmo, obtém-se uma função linear em x:

$$f_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log P(\omega_k)$$

O classificador atribui x à classe com maior $f_k(x)$. As fronteiras de decisão são lineares, e o modelo é ótimo se as hipóteses gaussianas forem verdadeiras.

8.3. Linear Regression of Indicator Variables

Uma alternativa ao LDA consiste em usar **regressão linear** aplicada a variáveis indicadoras de classe. Para cada classe k, define-se uma função discriminante obtida por mínimos quadrados:

$$f_k(x) = \beta_{0k} + \beta_k^T x$$

A classe predita é aquela cujo $f_k(x)$ é máximo. No caso binário, as funções são simétricas, e basta usar uma única função discriminante. Esta abordagem estabelece uma ligação direta entre a regressão linear e a estimativa de $P(\omega|x)$, embora possa gerar probabilidades fora do intervalo $[0, 1]$.

8.4. Logistic Regression

A **regressão logística** resolve o problema anterior, modelando explicitamente as probabilidades entre 0 e 1. No caso binário ($y \in \{0, 1\}$):

$$P(y = 1|x) = \frac{1}{1 + e^{-x^T \beta}}$$

A fronteira de decisão é linear ($x^T \beta = 0$), mas o modelo é probabilístico e mais robusto. Os parâmetros β são estimados por **máxima verosimilhança**, maximizando a log-likelihood:

$$\mathcal{L}(\beta) = \sum_{i=1}^{n} \left[y^{(i)} \log P(y^{(i)}|x^{(i)}) + (1 - y^{(i)}) \log(1 - P(y^{(i)}|x^{(i)})) \right]$$

A otimização é feita iterativamente por métodos baseados em gradiente (como *gradient ascent*).

8.5. Softmax Regression (Multiclass Logistic Regression)

Quando existem mais de duas classes ($K > 2$), a regressão logística generaliza-se para o modelo **Softmax**, no qual a probabilidade de pertencer à classe k é dada por:

$$P(y = k|x) = \frac{e^{x^T \beta_k}}{\sum_{j=1}^{K} e^{x^T \beta_j}}$$

A decisão final é:

$$f(x) = \arg\max_k P(y = k|x)$$

Tal como na regressão logística binária, os parâmetros β_k são ajustados por máxima verosimilhança. Este modelo assegura que todas as probabilidades são positivas e somam 1, sendo amplamente usado em redes neuronais e classificadores modernos.

8.6. Comparação e Intuição

- **LDA** é um modelo *gerativo*, baseado em suposições sobre as distribuições $p(x|\omega_k)$.
- **Logistic** e **Softmax regression** são modelos *discriminativos*, que aprendem diretamente a fronteira de decisão sem modelar $p(x)$.
- Ambos produzem fronteiras **lineares**, mas a regressão logística é mais flexível e menos dependente de hipóteses estatísticas.
- Em conjuntos de dados grandes e de classes não gaussianas, o **softmax** é geralmente preferido.
- O **LDA**, por outro lado, é interpretável e estatisticamente ótimo sob as suas hipóteses.

8.7. Síntese Geral

Os linear classifiers são modelos que separam as classes por meio de fronteiras de decisão lineares no espaço das variáveis de entrada. O objetivo é encontrar uma função que atribua a classe correta a cada observação. Admite-se, ainda, que as classes podem ser separadas por planos lineares.

Um classificador linear define, para cada classe, sua função discriminante. A observação pertencerá à classe que apresentar maior função discriminante. A fronteira é composta pelos pontos comuns a ambas as funções discriminantes.

O LDA (Linear Discriminant Analysis) é um modelo probabilístico baseado na regra de Bayes. É assumido que as distribuições condicionais de cada classe são gaussianas, com a mesma matriz de covariância. O modelo é ótimo se as hipóteses de normalidade forem verdadeiras.

Podemos, em vez do LDA, utilizar a regressão linear aplicada a variáveis indicadoras de classe. Para cada classe, define-se uma função discriminante obtida por meio de least squares. Num caso binário, as funções são simétricas e basta utilizar uma função discriminante. Mas existe um problema: este método pode gerar valores fora do intervalo [0, 1]. Este problema é resolvido por meio de regressão logística, que modela as probabilidades entre 0 e 1. Quando há mais de duas classes, a regressão logística generaliza-se para o modelo Softmax.

9. Support Vector Machines

As **Support Vector Machines (SVM)** foram introduzidas por Vapnik e Chervonenkis (1963) como modelos de classificação supervisionada. Inicialmente desenvolvidas para problemas binários lineares, foram mais tarde estendidas para fronteiras não lineares e regressão. O seu princípio base é encontrar o **hiperplano que melhor separa os dados**, maximizando a margem entre as classes.

9.1. Conceito e Motivação

O objetivo de uma SVM é dividir um conjunto de dados em duas regiões correspondentes às classes, através de um **hiperplano de decisão** cuidadosamente escolhido. O processo é desenvolvido em três etapas principais:

1. Classificadores lineares com *hard margin*;
2. Classificadores lineares com *soft margin*;

3. Extensão para classificadores não lineares.

9.2. Hiperplanos e Classificação Linear

Um **hiperplano** em $\mathbb{R}^p$ é definido pela equação:

$$x \cdot w + b = 0$$

onde w é o vetor normal e b o termo de desvio. A distância do hiperplano à origem é $|b|/\|w\|$. No contexto de classificação binária:

$$\hat{y} = \text{sign}(x \cdot w + b), \quad y \in \{-1, +1\}.$$

9.3. Caso I: Dados Linearmente Separáveis (Hard Margin)

Assume-se que os dados podem ser separados sem erros por um hiperplano. Entre todos os possíveis hiperplanos separadores, o SVM escolhe aquele que **maximiza a margem**, isto é, a distância entre as amostras mais próximas das duas classes.

Esses pontos mais próximos são denominados **support vectors**, e as retas (ou planos) que os contêm são os **margin hyperplanes**:

$$x^{(i)} \cdot w + b = +1 \quad \text{se } y^{(i)} = +1,$$

$$x^{(i)} \cdot w + b = -1 \quad \text{se } y^{(i)} = -1.$$

A margem é dada por:

$$\text{margem} = \frac{2}{\|w\|}.$$

9.4. Problema de Otimização (Hard Margin)

Para maximizar a margem, minimiza-se $\|w\|^2/2$ sujeito às restrições:

$$y^{(i)}(x^{(i)} \cdot w + b) \geq 1, \quad \forall i.$$

O problema é resolvido via **Lagrangiana**, introduzindo multiplicadores $\alpha_i \geq 0$:

$$L_P = \frac{1}{2}\|w\|^2 - \sum_i \alpha_i [y^{(i)}(x^{(i)} \cdot w + b) - 1].$$

As condições de Karush–Kuhn–Tucker (KKT) permitem derivar a **forma dual**:

$$\max_\alpha \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y^{(i)} y^{(j)} (x^{(i)} \cdot x^{(j)}),$$

com restrições:

$$\alpha_i \geq 0, \quad \sum_i \alpha_i y^{(i)} = 0.$$

O vetor normal é calculado por:

$$w = \sum_i \alpha_i y^{(i)} x^{(i)},$$

e o classificador final é:

$$f(x) = \text{sign}(x \cdot w + b).$$

Apenas os exemplos com $\alpha_i > 0$ contribuem para w; estes são os **support vectors**.

9.5. Caso II: Dados Não Linearmente Separáveis (Soft Margin)

Quando os dados não são perfeitamente separáveis, admite-se um pequeno número de erros. Introduzem-se variáveis de folga $\xi_i \geq 0$, que medem o grau de violação da margem. O novo problema de otimização é:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i,$$

sujeito a:

$$y^{(i)}(x^{(i)} \cdot w + b) \geq 1 - \xi_i, \quad \forall i.$$

O parâmetro C controla o compromisso entre a largura da margem e o número de erros. Valores altos de C penalizam fortemente os erros (margem pequena), enquanto valores baixos permitem margens mais largas e maior tolerância.

9.6. Dual Form e Interpretação (Soft Margin)

Na forma dual:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2}\alpha^T H \alpha,$$

com

$$0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y^{(i)} = 0,$$

e $H_{ij} = y^{(i)} y^{(j)} (x^{(i)} \cdot x^{(j)})$.

As condições KKT identificam:

- $\alpha_i > 0$: ponto é um **support vector**;
- $\alpha_i = C$: ponto **viola a margem** (está mal classificado).

9.7. Caso III: Classificadores Não Lineares e Kernel Trick

Para dados com fronteiras não lineares, mapeia-se o espaço original para um **espaço de features de dimensão superior**:

$$\tilde{x} = \phi(x),$$

onde se torna possível encontrar um hiperplano linear.

O truque do kernel consiste em substituir o produto interno $(x^{(i)} \cdot x^{(j)})$ por uma **função kernel**:

$$k(x^{(i)}, x^{(j)}) = \phi(x^{(i)}) \cdot \phi(x^{(j)}),$$

evitando calcular explicitamente $\phi(x)$.

9.8. Kernels Mais Comuns

- **Linear**: $k(x^{(i)}, x^{(j)}) = x^{(i)T} x^{(j)}$
- **RBF (Radial Basis Function)**: $k(x^{(i)}, x^{(j)}) = e^{-\frac{\|x^{(i)} - x^{(j)}\|^2}{2\sigma^2}}$
- **Polinomial**: $k(x^{(i)}, x^{(j)}) = (x^{(i)T} x^{(j)} + a)^b$

O kernel RBF é o mais utilizado, e os parâmetros (como σ ou C) são ajustados via **cross-validation**.

9.9. Extensão para Problemas Multiclasse

As SVM são originalmente **binárias**, mas podem ser adaptadas para $K > 2$ classes através de duas estratégias principais:

- **One-vs-One (OvO)**: treina-se um classificador para cada par de classes ($K(K-1)/2$ modelos); a decisão final é feita por votação.
- **One-vs-Rest (OvR)**: treina-se um classificador por classe, comparando cada uma contra todas as restantes; escolhe-se a classe com maior pontuação.

9.10. Síntese Geral

Tipo de SVM	Característica Principal	Função Objetivo	Observações
Hard Margin	Dados perfeitamente separáveis	$\min \frac{1}{2}\|w\|^2$	Sem erros, margem máxim
Soft Margin	Dados com ruído	$\min \frac{1}{2}\|w\|^2 + C\sum \xi_i$	Compromisso entre margen
Kernel SVM	Dados não lineares	Usa $k(x^{(i)}, x^{(j)})$	Permite fronteiras complex
Multiclasse	$K > 2$ classes	OvO ou OvR	Generalização binária

Table 6: Resumo das variantes de SVM.

10. Decision Trees and Random Forests

As **árvores de decisão** são classificadores muito populares, pois permitem compreender facilmente o motivo de uma classificação. Cada decisão é feita através de uma sequência de perguntas sobre as features, até se chegar a um nó terminal (folha), onde é atribuída uma classe. Esta interpretabilidade é essencial em áreas como a medicina ou a economia, onde o "porquê" da decisão é tão importante quanto o resultado.

10.1. Conceito Geral

Uma **árvore de decisão** é composta por três tipos de nós:

- **Nó raiz**: onde a primeira divisão (split) é feita;
- **Nós internos**: correspondem a perguntas sobre as variáveis (ex.: "Outlook = Sunny?");
- **Folhas (leaves)**: associadas às classes finais.

Cada nó contém um subconjunto dos dados de treino. Quando todas as amostras num nó pertencem à mesma classe, diz-se que o nó é **puro** e não é dividido novamente.

10.2. Exemplo: Good Days to Play Tennis

O exemplo clássico de Quinlan (1986) ilustra a construção de uma árvore para prever se um jogador (John) deve jogar ténis, com base em atributos como *Outlook*, *Humidity* e *Wind*. Apesar de existirem amostras com atributos idênticos mas rótulos diferentes (ruído nos dados), a árvore aprende regras lógicas claras, como:

- If Outlook = Overcast $\rightarrow$ Play = Yes
- If Outlook = Sunny and Humidity = High $\rightarrow$ Play = No

Cada divisão cria subconjuntos mais homogéneos até que todos os nós sejam puros ou suficientemente pequenos.

10.3. Treino e Dados em Cada Nó

Durante o treino, cada nó tem um subconjunto do conjunto de treino associado. Os exemplos são divididos de acordo com as respostas às perguntas feitas nos nós superiores. Se todas as amostras de um nó tiverem o mesmo rótulo, o nó é puro e torna-se uma folha. Caso contrário, o nó é dividido de forma a reduzir a **impureza**.

10.4. Medidas de Impureza

A impureza quantifica a heterogeneidade dos rótulos num nó. As medidas mais comuns são:

Erro de classificação:

$$i(m) = 1 - \max_k P(k|m)$$

Entropia:

$$i(m) = -\sum_{k=1}^{K} P(k|m) \log_2 P(k|m)$$

Índice de Gini:

$$i(m) = \sum_{k=1}^{K} P(k|m)(1 - P(k|m))$$

Nos problemas binários, tanto a entropia como o índice de Gini são funções suaves da proporção $P(k = 1|m)$, sendo geralmente preferidas ao erro simples.

10.5. Treino Ótimo da Árvore

O objetivo é minimizar a **impureza total da árvore**:

$$I(T) = \sum_{m \in \tilde{T}} P(m)\, i(m)$$

onde $P(m)$ é a fração de exemplos no nó m e $\tilde{T}$ é o conjunto de folhas.

Em teoria, deveríamos gerar todas as árvores possíveis e escolher a que minimiza $I(T)$, mas isso é computacionalmente impraticável. Na prática, usam-se estratégias de crescimento recursivo (*greedy search*): em cada nó escolhe-se o split que mais reduz a impureza.

10.6. Avaliação e Poda (Pruning)

Árvores profundas podem sobreajustar os dados. Para melhorar a generalização, aplicam-se técnicas de **poda**:

- **Poda por validação**: substitui-se subárvores por folhas se a sua remoção não aumentar o erro de validação;
- **Poda bottom-up**: testa-se cada nó de baixo para cima, removendo subárvores se o desempenho não piorar.

A generalização pode também ser estimada sem conjunto de validação, através do **limite de Wilson** (estimador conservador do erro esperado):

$$e = f + \frac{z_\alpha^2}{2N} + z_\alpha \sqrt{\frac{f}{N} - \frac{f^2}{N} + \frac{z_\alpha^2}{4N^2}}.$$

10.7. Extensão para Atributos Numéricos

Embora inicialmente desenhadas para dados categóricos, as árvores podem lidar com variáveis contínuas. Neste caso, os **splits são definidos por limiares (thresholds)**. Durante o treino:

- Todos os possíveis limiares são testados para cada variável;
- Escolhe-se o limiar que maximiza a redução da impureza;
- O processo repete-se recursivamente em cada novo nó.

O limiar ótimo t^* é:

$$t^* = \arg\min_t \mathrm{Imp}(t)$$

onde $\mathrm{Imp}(t)$ é a impureza ponderada das partições esquerda e direita.

10.8. Árvores de Regressão

As árvores também podem ser usadas em problemas de **regressão**, onde $y \in \mathbb{R}$. A impureza de um nó é medida pela soma dos erros quadráticos (SSE):

$$SSE(A) = \sum_{i \in A} (y_i - \bar{y}_A)^2$$

O split ideal é o que minimiza:

$$SSE(L) + SSE(R)$$

Cada folha prevê o valor médio $\bar{y}_A$ das amostras nesse nó. O resultado é uma função **piecewise-constant**, com regiões associadas a intervalos de valores de x.

10.9. Vantagens e Desvantagens

Vantagens:

- Simples e rápidos de treinar e prever;
- Altamente interpretáveis (regras explícitas);
- Lidam bem com variáveis categóricas e numéricas;
- Capturam relações não lineares e interações.

Desvantagens:

- Elevada variância (instabilidade com pequenas alterações nos dados);
- Propensão ao overfitting (árvores muito profundas);
- Fronteiras de decisão são retas alinhadas com os eixos (pouco flexíveis);
- Importância das features pode ser instável.

10.10. Random Forests

O **Random Forest** é um método de *ensemble* que combina múltiplas árvores de decisão, reduzindo a variância e melhorando a generalização. Cada árvore é treinada sobre um subconjunto aleatório dos dados (obtido por *bootstrap*) e das features.

O algoritmo é:

1. Gerar B conjuntos de treino $T^{(i)}$ via *bootstrap*;
2. Treinar uma árvore em cada conjunto $T^{(i)}$, escolhendo aleatoriamente um subconjunto de features em cada nó (*random subspace*);
3. Calcular as distribuições a posteriori $P^{(i)}(y = k|x)$ de cada árvore;
4. Agregar as previsões:

$$\hat{P}(y = k|x) = \frac{1}{B} \sum_{i=1}^{B} P^{(i)}(y = k|x).$$

10.11. Intuição e Comparação

As árvores de decisão são classificadores hierárquicos que organizam decisões em forma de árvore, onde cada divisão é baseada em condições sobre as features. Cada amostra percorre uma sequência de perguntas até atingir um nó terminal (folha), onde é atribuída uma classe. Uma árvore de decisão é composta por três tipos de nós: o nó raiz, onde ocorre o primeiro *split*; os nós internos, que correspondem a perguntas sobre as variáveis; e as folhas, que representam as classes finais.

Durante o treino, cada nó contém um subconjunto do conjunto de treino. As amostras são divididas conforme as respostas às perguntas feitas nos nós anteriores. Se todas as amostras de um nó tiverem o mesmo rótulo, o nó é dito puro e torna-se uma folha. Caso contrário, é dividido de modo a reduzir a impureza, isto é, a heterogeneidade dos rótulos.

As medidas de impureza mais comuns são o erro de classificação, a entropia e o índice de Gini, e o objetivo do treino é minimizar a impureza total da árvore. As árvores também podem ser utilizadas em problemas de regressão, onde cada folha prevê um valor numérico médio.

As árvores de decisão são simples, rápidas e interpretáveis, mas sofrem de elevada variância, o que as torna sensíveis a pequenas mudanças nos dados. As Random Forests resolvem este problema combinando múltiplas árvores de decisão treinadas sobre diferentes subconjuntos aleatórios de dados e de variáveis (técnica de *bagging*). Esta abordagem reduz a variância e melhora a capacidade de generalização. O algoritmo mais comum para construir cada árvore dentro da floresta é baseado em variantes dos métodos CART ou ID3, ajustando os *splits* por ganho de informação ou índice de Gini.

Método	**Ideia Principal**	**Vantagem**	**Limitaçã**
Decision Tree	Regras hierárquicas de decisão	Interpretável	Alta variâ
Bagging	Média de modelos independentes	Reduz variância	Menor int
Random Forest	Bagging + subamostragem de features	Menor correlação entre árvores	Menos int

Table 7: Comparação entre Decision Trees e Random Forests.

11. Boosting

O **Boosting** é uma estratégia de *ensemble* que constrói um preditor como uma soma ponderada de múltiplos **weak learners**, geralmente árvores de decisão simples. Cada modelo é treinado sequencialmente, de modo que cada novo modelo tenta corrigir os erros cometidos pelos anteriores. Esta abordagem permite combinar classificadores fracos num modelo altamente preciso.

11.1. Conceito e Intuição

O modelo final é uma combinação aditiva de M classificadores fracos:

$$F_M(x) = \sum_{m=1}^{M} \alpha_m f_m(x)$$

onde α_m é o peso atribuído ao classificador $f_m(x)$. Cada novo classificador é treinado com base no desempenho dos anteriores, concentrando-se nos exemplos mais difíceis. Enquanto o **Bagging** reduz a variância (através de modelos paralelos e independentes), o **Boosting** reduz o **viés**, pois melhora iterativamente o ajuste do modelo.

11.2. Diferenças entre Bagging e Boosting

- **Bagging:** treina modelos em paralelo sobre subconjuntos de dados obtidos por *bootstrap*. As previsões são combinadas por média (regressão) ou voto (classificação). Reduz variância e evita sobreajuste.
- **Boosting:** treina modelos de forma sequencial, atribuindo mais peso aos exemplos mal classificados. Reduz o viés, mas pode sobreajustar se não for regularizado.

Exemplos clássicos incluem o **AdaBoost**, o **Gradient Boosting** e o **XGBoost**.

11.3. AdaBoost (Adaptive Boosting)

O AdaBoost foi o primeiro algoritmo de Boosting amplamente utilizado, proposto por Freund e Schapire (1997). É um método iterativo para classificação binária, em que cada instância de treino x_i recebe um peso w_i.

1. Inicializam-se pesos uniformes $w_i^{(1)} = 1/n$.
2. Para cada iteração $m = 1, \ldots, M$:
   (a) Treina-se um classificador fraco $f_m(x) \in \{-1, +1\}$ com base nos pesos.
   (b) Calcula-se o erro ponderado:
   $$\varepsilon_m = \frac{\sum_i w_i^{(m)} 1[y_i \neq f_m(x_i)]}{\sum_i w_i^{(m)}}$$
   (c) Define-se o peso do classificador:
   $$\alpha_m = \frac{1}{2} \ln \frac{1 - \varepsilon_m}{\varepsilon_m}$$
   (d) Atualizam-se os pesos das amostras:
   $$w_i^{(m+1)} = w_i^{(m)} \exp(-\alpha_m y_i f_m(x_i))$$

(e) Normalizam-se os pesos para que somem 1.

3. O classificador final é:

$$F(x) = \text{sign}\left(\sum_{m=1}^{M} \alpha_m f_m(x)\right)$$

O AdaBoost pode ser interpretado como um método de minimização da **exponential loss**:

$$L(y, F(x)) = e^{-yF(x)}.$$

A atualização dos pesos aumenta a importância dos exemplos incorretamente classificados, incentivando o modelo seguinte a corrigi-los.

11.4. Relação entre AdaBoost e Gradient Boosting

O AdaBoost pode ser visto como um caso particular do **Gradient Boosting**, onde a perda é exponencial e a taxa de aprendizagem $\eta = 1$. Enquanto o AdaBoost repondera amostras, o Gradient Boosting interpreta o processo como uma descida de gradiente funcional sobre uma função de perda arbitrária e diferenciável.

11.5. Gradient Boosting

O **Gradient Boosting** generaliza o Boosting ao interpretar o modelo como uma sequência de aproximações sucessivas de um gradiente negativo no espaço de funções.

A ideia é construir um modelo aditivo:

$$F_M(x) = F_{M-1}(x) + \eta f_M(x)$$

onde η é a **taxa de aprendizagem** e $f_M(x)$ é um novo estimador ajustado aos resíduos (ou gradientes) da iteração anterior.

Algoritmo genérico:

1. Inicializar $F_0(x) = \arg\min_c \sum_i L(y_i, c)$.
2. Para $m = 1, \ldots, M$:
   (a) Calcular os **pseudo-resíduos**:
   $$r_i^{(m)} = -\frac{\partial L(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)}.$$
   (b) Ajustar um regressor $f_m(x)$ aos pares $(x_i, r_i^{(m)})$.
   (c) Atualizar:
   $$F_m(x) = F_{m-1}(x) + \eta f_m(x).$$

Esta formulação permite aplicar o Boosting a qualquer função de perda diferenciável (ex.: erro quadrático, log-loss, exponencial, Huber).

11.6. Intuição Geométrica

O Gradient Boosting pode ser interpretado como uma descida de gradiente no espaço funcional. Em cada iteração, é dada um passo na direção do gradiente negativo da perda $L(F)$:

$$F_m = F_{m-1} - \eta \nabla L(F_{m-1}).$$

Na prática, o gradiente é aproximado pelos resíduos ponto a ponto, e um pequeno modelo (geralmente uma árvore de regressão rasa) é ajustado para os reproduzir. Isto permite capturar não linearidades e interações complexas entre variáveis.

11.7. Exponential Loss e Margem de Classificação

Em classificação binária ($y \in \{\pm 1\}$), define-se a margem $m = yF(x)$. A perda exponencial:

$$L(y, F) = e^{-yF}$$

penaliza fortemente amostras incorretamente classificadas ($yF < 0$) e recompensa classificações confiantes ($yF > 0$). Mesmo após atingir erro de treino zero, o treino pode continuar a aumentar a margem, melhorando a confiança das previsões.

11.8. XGBoost (Extreme Gradient Boosting)

O **XGBoost** é uma versão otimizada e regularizada do Gradient Boosting, que melhora o desempenho e a estabilidade.

Principais características:

- **Otimização de segunda ordem:** usa gradientes (g_i) e Hessianos (h_i) para atualizar mais rapidamente:

$$\ell(y_i, \hat{y}_i + f_m(x_i)) \approx \ell(y_i, \hat{y}_i) + g_i f_m(x_i) + \frac{1}{2} h_i f_m(x_i)^2.$$

- **Regularização:** adiciona penalizações à complexidade das árvores:

$$\Omega(f_m) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} v_j^2,$$

  onde T é o número de folhas e v_j os valores das folhas.

- **Shrinkage e Subsampling:** controla o sobreajuste e reduz a variância.
- **Alta eficiência computacional:** paralelização e otimização no cálculo dos gradientes.

Vantagens principais:

- Elevada precisão em dados tabulares;
- Controla o equilíbrio entre viés e variância;
- Suporta dados com *missing values* e diferentes tipos de variáveis;
- Excelente relação desempenho/interpretabilidade.

Limitações:

- Maior número de hiperparâmetros;
- Requer tuning cuidadoso da profundidade, taxa de aprendizagem e regularização;
- Menos interpretável que uma única árvore de decisão.

11.9. Conclusão

O Boosting é uma das estratégias mais poderosas em Aprendizagem Automática moderna. Transforma classificadores fracos em modelos robustos através de aprendizagem sequencial. O AdaBoost introduziu o conceito; o Gradient Boosting generalizou-o para qualquer função de perda; e o XGBoost levou-o a um nível superior com otimização de segunda ordem e regularização. Em particular, o XGBoost tornou-se o método de referência para dados estruturados, equilibrando **precisão**, **robustez** e **eficiência computacional**.

Problem A

Explain briefly the backpropagation algorithm applied to the training of neural networks. Discuss the ability to converge to the global minimum of the loss function. For cases in which the evolution of the algorithm is slow, what can be done to accelerate it?

Solution: A neural network is a map from input data x to output data y. It can be represented as a function of the data x, y, and of a set of learnable weights w: $\hat{y} = f(x, w)$. To learn the weights, a loss function is defined to express errors between the predictions $\hat{y}$ and the desired outputs y. The weights are adapted by gradient descent (or variants). The backpropagation algorithm is an efficient method to compute the gradient of the loss function with respect to the weights. The gradient is calculated using the chain rule of calculus, which allows the error to be propagated back through the network. The algorithm is divided into two main steps: the forward pass, in which the input data is propagated through the network to obtain the output, and the backward pass, in which the error is propagated back through the network to calculate the gradient of the loss function with respect to the weights. In the forward step, for each training pattern presented to the network, the backpropagation algorithm computes and stores the activations of the neurons and the derivatives of the activation functions. Then, the partial derivative of the loss function for that pattern with respect to the neural network output is computed and "backpropagated" through the transpose network. The transpose network is the network obtained by the transposition of the graph of the neural network: the direction of the information flow is reversed from the output to the input, the weights are preserved, the activation functions are replaced by weights corresponding to their derivatives, the derivation points are transformed in sums, and the sum points in derivation points. After the information has flowed in the transpose network, the values present at the derivation points are stored. These values (present at the output of a weight edge) are multiplied by the input of the weight edge, to compute the partial derivative of the loss function with respect to that weight and that pattern. These gradients can be used immediately to update the weights (online mode) or accumulated for a batch of data before updating the weights (batch mode). The backpropagation algorithm gives no warranties to converge to the global minimum, or even to converge at all. To improve its

convergence properties, the adaptation step parameter must be carefully adjusted, and some acceleration methods can be used, like momentum or adaptive steps.

—

Problem B

Consider a classification problem in which we want to classify a vector of continuous features $x = [x_1, x_2, \cdots, x_p]^T$ in a set of discrete classes $y \in \{\omega_0, \omega_1, \cdots, \omega_{k-1}\}$ using the Random Forest method and the ID3 algorithm. What are the main steps required in the training stage, and for classifying a new feature at test time? Explain what is the Bagging algorithm and how it is used in the Random Forest context. What are the main differences in training a Random Forest compared to a single decision tree?

Solution: Random Forest is a learning method that combines multiple decision trees. It is typically trained using the Bagging algorithm, which creates different subsets of the training data via bootstrap sampling (random sampling with replacement) and trains a decision tree on each subset. The final prediction is obtained by aggregating the predictions of all trees. The main steps required in the training of a Random Forest are:

- Randomly select n samples from the training dataset with replacement (bootstrap sampling).
- Grow a decision tree from the selected samples. At each node, a random subset of features is considered for splitting.
- Repeat the previous steps m times to create m decision trees.
- The final prediction is obtained by aggregating the predictions of all trees (majority voting for classification).

To grow each individual decision tree, the ID3 algorithm is used to recursively partition the data based on the selected features. To consider continuous input data, not only the features but also possible thresholds to split the continuous features must be considered. The features and threshold values are chosen to maximize the impurity reduction at each node of the tree. The ID3 algorithm uses entropy as a measure of impurity. For classifying a new sample at test time, the sample is passed through all trees, and the final prediction is obtained by aggregating the predictions of all trees. The main difference in training a Random Forest compared to a single decision tree is that a Random Forest uses a random subset of features at each node for splitting, while a single decision tree considers all features for splitting.

Problem C

Consider a regression problem where training data contains a small proportion of outliers. Describe two strategies to mitigate the outliers' influence on the estimation of the regression model parameters.

Solution:

- **Strategy 1 – iterative outlier removal:** In this strategy, a regression model is trained iteratively. Initially, the regression is computed using all the training data. Then, the residuals (difference between observed and predicted values) are

calculated, and a few outliers (or just one) are identified as the ones with the largest absolute value of the residual. The identified outliers are removed, and the regression is re-computed using only the inliers. The entire process is repeated until a maximum number of iterations is reached, or no more outliers are identified.

- **Strategy 2 – RANSAC (Random Sample Consensus):** In RANSAC, n points are randomly selected from the dataset (n is the minimum needed to determine the regression model) and a regression model is fitted to the randomly selected points. Then the estimated model is evaluated on the entire dataset to count the number of inliers. A data point is considered an inlier if the associated residual is below a predefined threshold. This process is repeated many times, and the model with the largest number of inliers is selected as the best model.

—

Problem D

One of the biggest challenges of supervised machine learning problems is the availability of annotated/labeled data. Often, we have access to a small pool of labeled data and a bigger one with unlabeled data. Describe how you could explore the unlabeled data to improve a model that was trained with the small pool of labeled data. You should also discuss the advantages and limitations of your strategy.

Solution: Adopt an iterative pseudo labeling + retraining/fine tuning strategy to improve the model. The first model trained on the small labeled dataset is used to label the samples. Then we select the samples with the highest confidence (e.g., class probabilities above a threshold) to augment the training set and retrain/fine tune the original model. We do this multiple times (pseudo labeling, sample selection, and retraining) to maximize the usage of unlabeled data until a stopping criterion is met (e.g., the performance no longer improves on a validation set or there are no pseudo labeled samples with a confidence above a defined threshold).

- **Advantages:** Ideally train more robust and generalizable models thanks to using more data. This is achieved without needing to actually label the data by hand, which is cumbersome and time consuming.
- **Disadvantages (at least two of the following):**
  - The initial model can be biased or poorly trained, which will affect the pseudo labels and potentially introduce noise and augment the bias.
  - There are no guarantees that the pseudo labels are actually correct. Although this can be mitigated by selecting only samples with the highest probabilities.
  - The unlabeled dataset may be from a different distribution to that of the labeled one, which can lead to incorrect pseudo labels.

Problem E

MNIST is large dataset of 28×28 pixel images for training digit classifiers from images (see Figure 2). Consider that images are binarized, i.e. each pixel either has value 0 (background - white) or 1 (foreground - black). Discuss

with as much detail as possible how to create a Naive Bayes classifier for this problem.

We have 10 classes (the digits) $y \in \{0, \cdots, 9\}$, and $m = 28 \times 28 = 784$ features (the value of each pixel) $x_i \in \{0, 1\}$, $i = 1, \cdots, m$, where i encodes the location (line, column) of a pixel in the image. The Naive Bayes classifier is a Bayes classifier in which the features are assumed conditionally independent for each class:

$$\hat{y} = \arg\max_k P(y = k|x_1, x_2, \cdots, x_m) = \arg\max_k p(x_1, x_2, \cdots, x_m|y = k)P(y = k)$$

$$= \arg\max_k \prod_{i=1}^{m} p(x_i|y = k)P(y = k)$$

To create a Naive Bayes classifier for this problem, we need to estimate, from the training set, the densities $p(x_i|y = k)$ and the distribution $P(y)$.

$P(y)$ is a categorical distribution with $P(y = k) = p_k$ in which p_k can be approximated by the fraction of digits k in the whole dataset, $p_k = \frac{N_k}{N}$. N_k is the number of images of digit k in the dataset, and N is the total number of images in the dataset. We have to compute 10 numbers p_k.

To represent $p(x_i|y = k)$, because the values of the pixels can only have values $x_i = 0$ or $x_i = 1$, the most suitable density is the Bernoulli distribution

$$p(x_i|y = k) = (f_{i,k})^{x_i}(1 - f_{i,k})^{1-x_i},$$

in which parameters $f_{i,k}$ can be approximated by the fraction of foreground pixels in location i ($x_i = 1$) over all images of class k:

$$f_{i,k} = \frac{\sum_{n \in \Omega_k} x_i^{(n)}}{N_k},$$

where Ω_k is the set of images of digit k.

To prevent cases in which $p(x_i|y = k) = 0$, that would make the method not robust to noise, it is often applied the technique of **Laplace smoothing**, in which the $f_{i,k}$ are computed as

$$f_{i,k} = \frac{\sum_{n \in \Omega_k} x_i^{(n)} + \alpha}{N_k + 2\alpha},$$

where α is a positive number. We need to compute 784×10 numbers $f_{i,k}$.

Finally, at test time, given a new image with pixels x'_i, the classifier output is:

$$\hat{y} = \arg\max_k p_k \prod_{i=1}^{m} (f_{i,k})^{x'_i}(1 - f_{i,k})^{1-x'_i}$$

—

Problem F

In a soft-margin SVM it is necessary to define the soft-margin penalty parameter, typically denoted C. Discuss in detail a possible strategy to choose a value for C that improves the performance of the algorithm in unseen data.

To select the hyperparameter C we could proceed as follows: i) specify a set of C admissible values (grid search) and, for each C value perform the following operations: train and evaluate the model; ii) select the best model (C value); iii) at the end retrain the SVM for the selected C, using all the training patterns.

To be fair, model training and evaluation performed in step i) should not be done using the same set of data. One approach consists of splitting the training set into a smaller training set (with 80% of patterns) and a validation set with the remaining patterns. The first set would be used for training and the other for model evaluation. If the training set is small, there are alternative procedures, e.g., n-fold cross validation. In this methodology the training set is divided in n subsets, and n different models are created in the possible combinations of $n-1$ subsets and evaluated in the remaining subset. Then, the average score is computed and used for step ii).

The model evaluation criterion required in step ii) would be the number of classification errors in the validation set.

Problem G

Some linear classifiers can represent non-linear separating boundaries on the original input space through transformations of the input, typically increasing its dimensionality. However, with the Kernel Trick, it is possible to implement non-linear separating boundaries with linear classifiers without explicitly performing computations in the (higher dimension) transformed space. In the context of Support Vector Machines, explain what is a Kernel function (what are its inputs and outputs), what operations it performs, and how it implements the Kernel Trick. Include mathematical formulas, diagrams or other illustrations you may find useful to accompany your explanation. Give some examples of known Kernel functions.

Solution: A kernel function operates on two input samples and computes a scalar number that represents their similarity or a measure of how close or far they are in the input space $z_{i,j} = K(x_i, x_j)$. When input samples are non-linearly transformed in a higher dimension feature space $y = \phi(x)$, the features y may be more effectively classified by an SVM than in the original space x. As the solution of an SVM classifier only depends on the inner product of the vectors to classify, the increase of the dimension of the vector space would increase the computational cost of the solution. However, instead of directly performing the inner product of the high dimension feature vectors, $\langle \phi(x_i), \phi(x_j) \rangle$, we can replace this operation with a Kernel function $K(x_i, x_j)$ that obtains the same result but with fewer computations. This is called the **Kernel Trick**.

Popular kernel functions are:

$$\text{Linear: } K(x_i, x_j) = x_i^T x_j,$$

(feature space is the same as the original space),

$$\text{Polynomial: } K(x_i, x_j) = (x_i^T x_j + c)^d,$$

and

$$\text{Radial Basis Function (Gaussian): } K(x_i, x_j) = \exp(-\gamma ||x_i - x_j||^2).$$

—

Problem H

Explain what convolutional neural networks are and what principles they are based on. Discuss in which cases they are appropriate techniques and in which cases they are not.

Convolutional neural networks (CNNs) are inspired by the human visual system, which has two important properties: i) **invariance to translation** (the system is capable of recognizing objects despite their location), and ii) **local processing** (inspired by receptive fields: neurons in the cortex are activated by a small neighborhood in the field of view).

This suggests the use of convolutions with small kernels where each neuron shares the same weights with the others. CNNs are organized into a sequence of layers that typically perform convolutions followed by nonlinearities, pooling (sub-sampling) layers and end with one or more fully connected layers.

CNNs are powerful tools if the input signal has spatial or temporal organization (e.g., image, speech) but it makes no sense to apply them if the network input is a collection of features without such organization.

Problem I

Suppose that we have a training set for regression with n examples and p features with $p > n$. Describe why regularization is necessary and what are the main differences between ridge regression and lasso regression.

Solution: When the number of features p is greater than the number of training examples n, the regression problem is said to be underdetermined. In this case, there are infinitely many possible solutions for the parameter vector β that perfectly fit the training data, which leads to overfitting and poor generalization to unseen data. The estimated model becomes extremely sensitive to small variations or noise in the training set, resulting in high variance.

Regularization methods add a penalty term to the loss function to constrain the magnitude or structure of the coefficients, improving stability and generalization. The most common approaches are **Ridge Regression** and **Lasso Regression**.

In ordinary least squares regression, the objective function is:

$$J(\beta) = \|y - X\beta\|_2^2.$$

Ridge Regression (L2 regularization) adds a penalty proportional to the squared magnitude of the coefficients:

$$J_{\text{ridge}}(\beta) = \|y - X\beta\|_2^2 + \lambda\|\beta\|_2^2,$$

where $\lambda > 0$ controls the strength of regularization. Ridge regression shrinks all coefficients toward zero but never exactly to zero, reducing variance and handling multicollinearity well.

Lasso Regression (L1 regularization) uses instead the sum of the absolute values of the coefficients:

$$J_{\text{lasso}}(\beta) = \|y - X\beta\|_2^2 + \lambda\|\beta\|_1 = \|y - X\beta\|_2^2 + \lambda \sum_i |\beta_i|.$$

Lasso not only shrinks coefficients but can set some of them exactly to zero, effectively performing **feature selection** by keeping only the most relevant predictors.

Main differences:

- Ridge keeps all features but reduces their influence (continuous shrinkage);
- Lasso produces sparse solutions, eliminating irrelevant features;
- Ridge is preferable when many features are correlated, while Lasso is useful when only a few are expected to be important.

Regularization is therefore essential in high-dimensional settings ($p > n$) to ensure that the model is identifiable, numerically stable, and generalizes well to new data.

—

Problem J

Considering the case of binary classification tasks, comment on the relation between logistic regression models and neural networks. For instance, suppose we are given N distinct logistic regression models, trained for a given binary classification task, and decide to take a weighted ensemble of these models to make predictions. Can you express the ensemble in terms of a single artificial neural network and, if so, what would be its form?

Solution: A logistic regression model can be viewed as a simple neural network with a single neuron and a sigmoid activation function. For binary classification, the output is given by:

$$\hat{y} = \sigma(w^T x + b),$$

where $\sigma(s) = \frac{1}{1+e^{-s}}$ is the logistic (sigmoid) function.

If we have N distinct logistic regression models, each producing an output:

$$\hat{y}_i = \sigma(w_i^T x + b_i), \quad i = 1, 2, \ldots, N,$$

and we form a weighted ensemble of these models:

$$\hat{y}_{\text{ens}} = \sigma\left(\sum_{i=1}^{N} \alpha_i \hat{y}_i + \beta\right),$$

this ensemble can be expressed as a **two-layer neural network**:

- The **first layer** consists of N neurons (one per logistic model), each computing $z_i = \sigma(w_i^T x + b_i)$;
- The **second layer** is a single neuron that combines the z_i's linearly with weights α_i and applies a final sigmoid activation:

$$\hat{y} = \sigma\left(\sum_{i=1}^{N} \alpha_i z_i + \beta\right).$$

This structure corresponds to a standard feedforward neural network with one hidden layer of N sigmoid units and one output neuron with sigmoid activation.

In summary:

- Logistic regression $\Rightarrow$ single-layer (no hidden units) neural network;
- Weighted ensemble of N logistic regressions $\Rightarrow$ two-layer neural network;

- The first layer corresponds to the individual logistic models, and the second layer aggregates their outputs.

Therefore, an ensemble of logistic regressions can be interpreted as a **multi-layer perceptron (MLP)** with a single hidden layer, demonstrating how neural networks generalize logistic regression models.

Problem K

The Bayes Classifier is a well known method in data classification. Briefly explain:

- what is its main advantage?
- what is its main disadvantage?
- what is $P(y = i|x)$? (the notation is the one used in the lectures)

Solution: The Bayes Classifier assigns a new observation x to the class with the highest posterior probability:

$$\hat{y} = \arg\max_i P(y = i|x).$$

Main advantage: It is the **optimal classifier** in the sense that it minimizes the expected classification error (or Bayes risk), assuming that the true class-conditional densities $p(x|y)$ and prior probabilities $P(y)$ are known.

Main disadvantage: In practice, the true distributions $p(x|y)$ are rarely known and must be estimated from data. Estimating these densities accurately, especially in high-dimensional spaces, is difficult and often leads to poor performance (curse of dimensionality).

Interpretation of $P(y = i|x)$: $P(y = i|x)$ is the **posterior probability** that a sample with features x belongs to class i, computed using Bayes' theorem:

$$P(y = i|x) = \frac{p(x|y = i)P(y = i)}{p(x)},$$

where $p(x) = \sum_j p(x|y = j)P(y = j)$ ensures normalization.

—

Problem L

Decision trees trained with the ID3 algorithm tend to overfit the data. What can be done to improve the performance of the classifier?

Solution: Decision trees trained with the ID3 algorithm often overfit the training data because they grow until all examples are perfectly classified, creating overly complex trees that capture noise instead of general patterns.

To improve performance and generalization, the following strategies can be applied:

- **Pruning:** Reduce the size of the tree by removing branches that do not significantly improve performance on a validation set.
  - *Pre-pruning (early stopping):* Stop tree growth early when information gain is below a threshold or the node has too few samples.

  - *Post-pruning:* Grow the full tree, then remove branches that do not improve validation accuracy.

- **Cross-validation:** Use validation folds to choose the tree parameters (e.g., maximum depth, minimum number of samples per leaf).

- **Regularization parameters:** Limit the maximum depth, number of leaves, or minimum split size.

- **Ensemble methods:** Combine many trees to reduce variance, as in Bagging or Random Forests, or use Boosting techniques such as AdaBoost.

These approaches reduce variance and improve the tree's generalization ability, preventing it from memorizing the training data.

—

Problem M

Explain what is the rationale behind regularization methods. Give three examples of techniques that may benefit from the use of regularization.

Solution: The main rationale behind regularization is to prevent **overfitting** and improve the model's ability to generalize to unseen data by penalizing model complexity. Regularization adds a constraint or penalty term to the loss function to discourage large parameter values or overly flexible models.

Mathematically, if the original loss is $L(\theta)$, regularization modifies it as:

$$J(\theta) = L(\theta) + \lambda R(\theta),$$

where $R(\theta)$ is the regularization term (e.g., a norm of the parameter vector) and λ controls the strength of the penalty.

Regularization helps balance the trade-off between bias and variance — slightly increasing bias while substantially reducing variance.

Examples of techniques that benefit from regularization:

- **Linear and Logistic Regression:** Using L1 (Lasso) or L2 (Ridge) penalties to shrink coefficients and avoid overfitting.

- **Neural Networks:** Techniques such as weight decay (L2 regularization), dropout, and early stopping improve generalization and prevent large weight magnitudes.

- **Support Vector Machines:** The regularization parameter C in soft-margin SVMs controls the trade-off between maximizing the margin and minimizing classification error.

In all cases, regularization improves stability, reduces variance, and prevents the model from fitting noise in the training data.